

WILP, BITS Pilani

ZC416 MATHEMATICAL FOUNDATIONS FOR ML &amp; DS · COMPANION READER

# Gradient Descent

How a model finds its best parameters. Walk downhill, choose your step, and let a little noise help you scale.

Session 8 (Topic #9) · Read alongside the lecture slides · Every slide example worked out in full.

Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.

**How to use this companion.** The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation. Every slide example is worked out in full, step by step. Colour-coded boxes guide you. The **blue** “intuition” box states the core idea in one breath. The **amber** “everyday picture” box ties it to real life before any math. The **green** “worked example” box solves a problem with real numbers and no skipped steps. The **red** “watch out” box flags the common mistake. The **purple** “key takeaway” box is the one sentence to remember. Read it alongside the slides, in order.

## 1 The big idea: turning “fit a line” into “find a valley”

This lecture is about **continuous optimization**. That means tuning numbers until a score is as small as possible. There are two branches. **Constrained** optimization adds side rules the answer must obey. **Unconstrained** optimization, our topic, has none: any numbers are allowed. We start with the smallest real problem the slides give. Here is a tiny data table:

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| $x$ | 1   | 2   | 3   | 4   |
| $y$ | 3.1 | 4.9 | 7.3 | 9.1 |

The easiest model of  $y$  is a straight line,  $y = ax + b$ . Here  $a$  is the **slope** (how fast  $y$  grows with  $x$ ) and  $b$  is the **intercept** (the value at  $x = 0$ ). Which  $a$  and  $b$  are best? To answer that, we first need a score that says how bad any guess is.

### The intuition

Turn “find the best line” into “find the lowest point of a score”. Give every pair of knob settings a badness number. The best knobs are the ones sitting at the bottom of that badness landscape. All of machine learning training is this one move.

### ★ Everyday picture

Think of tuning an old radio with two dials to get the clearest signal. Each dial position produces some static. You listen (measure the badness), nudge a dial, and listen again. “Fitting a model” is exactly this: two dials ( $a$  and  $b$ ), one noise level (the loss). The analogy breaks in one way: the radio ends when it sounds fine, while we can measure badness exactly.

Now we build the score. Write  $\hat{y} = ax + b$  for the **predicted** value (the hat means “the model’s guess”). For data point  $i$ , the miss is the gap  $y_i - \hat{y}_i$ . Gaps can be positive or negative, and we do not want them cancelling out. So we square each gap and add them up. The result is the **loss function** from the slide:

$$L(a, b) = \sum_{i=1}^4 (y_i - \hat{y}_i)^2 = \sum_{i=1}^4 (y_i - (ax_i + b))^2. \quad (1)$$

Read the symbols slowly.  $\sum_{i=1}^4$  (the Greek capital sigma) means “add this up for  $i = 1, 2, 3, 4$ ”. Each term is one squared gap. Small  $L$  means the line hugs the points. Finding  $a$  and  $b$  that make  $L(a, b)$  smallest is an **optimization problem** — and that is the whole lecture in one sentence.

#### Worked example: scoring two candidate lines with the loss

We score the guess  $(a, b) = (1, 1)$  and the guess  $(a, b) = (2, 1)$  on the table above.

**Step 1. Predictions for  $(1, 1)$ .** The line is  $\hat{y} = x + 1$ , so  $\hat{y} = 2, 3, 4, 5$  at  $x = 1, 2, 3, 4$ .

**Step 2. Gaps for  $(1, 1)$ .**  $3.1 - 2 = 1.1$ ;  $4.9 - 3 = 1.9$ ;  $7.3 - 4 = 3.3$ ;  $9.1 - 5 = 4.1$ .

**Step 3. Square and add.**  $1.21 + 3.61 + 10.89 + 16.81 = \mathbf{32.52}$ . So  $L(1, 1) = 32.52$ .

**Step 4. Predictions for  $(2, 1)$ .** The line is  $\hat{y} = 2x + 1$ , so  $\hat{y} = 3, 5, 7, 9$ .

**Step 5. Gaps for  $(2, 1)$ .**  $0.1$ ;  $-0.1$ ;  $0.3$ ;  $0.1$ .

**Step 6. Square and add.**  $0.01 + 0.01 + 0.09 + 0.01 = \mathbf{0.12}$ . So  $L(2, 1) = 0.12$ .

**Step 7. Sense-check.** Both scores are positive, as sums of squares must be. The second line scores about 271 times better, and the picture agrees: it nearly threads the points.

The loss turns “which line looks better?” into plain arithmetic.

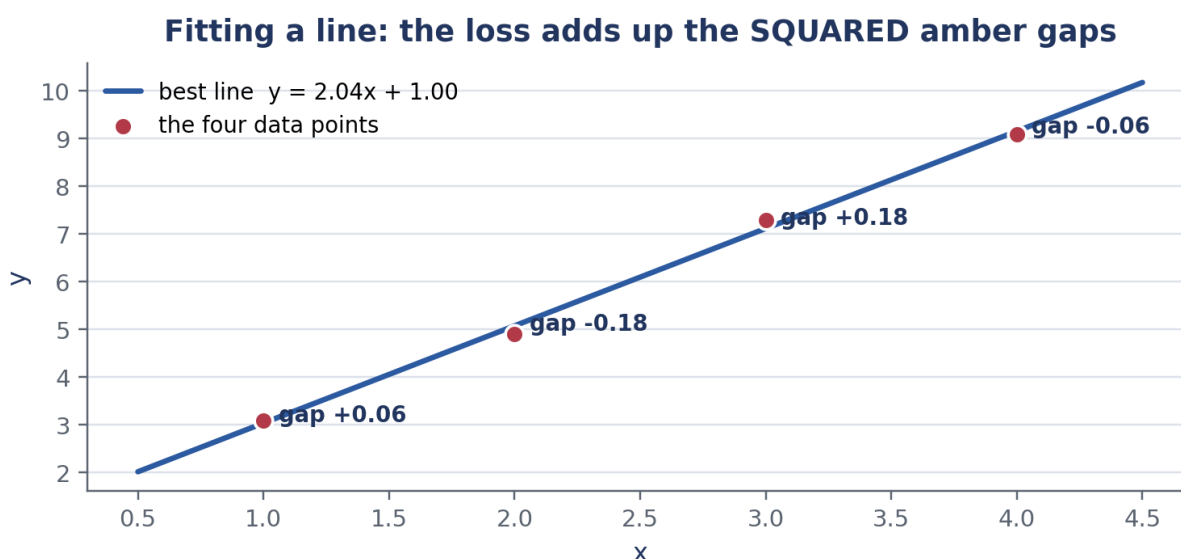


Figure 1: The four data points, the best line, and the gaps. How to read it: each dashed amber segment is one miss  $y_i - \hat{y}_i$ ; the loss is the sum of their squares, so shrinking the gaps shrinks the loss.

For this tiny problem we can even find the exact bottom by hand. At the lowest point of a smooth landscape, the ground is flat in every direction. Flat means both partial slopes are zero, so we set them to zero and solve.

 **Worked example: the exact best line, by setting slopes to zero**

We minimise  $L(a, b)$  over the four points.

**Step 1. Slope of  $L$  in  $a$ .** Differentiate each squared gap by  $a$  (chain rule: bring down the inner slope  $-x_i$ ):  $\frac{\partial L}{\partial a} = -2 \sum_{i=1}^4 x_i (y_i - (ax_i + b))$ .

**Step 2. Slope of  $L$  in  $b$ .** The inner slope in  $b$  is  $-1$ , so  $\frac{\partial L}{\partial b} = -2 \sum_{i=1}^4 (y_i - (ax_i + b))$ .

**Step 3. Set both to zero.** Divide out the  $-2$  and expand the sums, using  $\sum x_i = 10$ ,  $\sum x_i^2 = 30$ ,  $\sum y_i = 24.4$ ,  $\sum x_i y_i = 71.2$ :

$$30a + 10b = 71.2, \quad (2)$$

$$10a + 4b = 24.4. \quad (3)$$

**Step 4. Eliminate  $a$ .** Multiply the second equation by 3:  $30a + 12b = 73.2$ . Subtract the first:  $2b = 2$ , so  $b = 1.0$ .

**Step 5. Back-substitute.**  $10a = 24.4 - 4(1.0) = 20.4$ , so  $a = 2.04$ .

**Step 6. Sense-check.**  $L(2.04, 1.0) = 0.072$ , smaller than the 0.12 of our good guess in the last box. No pair beats it.

The trick used here — “the minimum is where the slopes vanish” — is the seed of the whole lecture.

### The loss $L(a, b)$ is a bowl — one best pair at the bottom

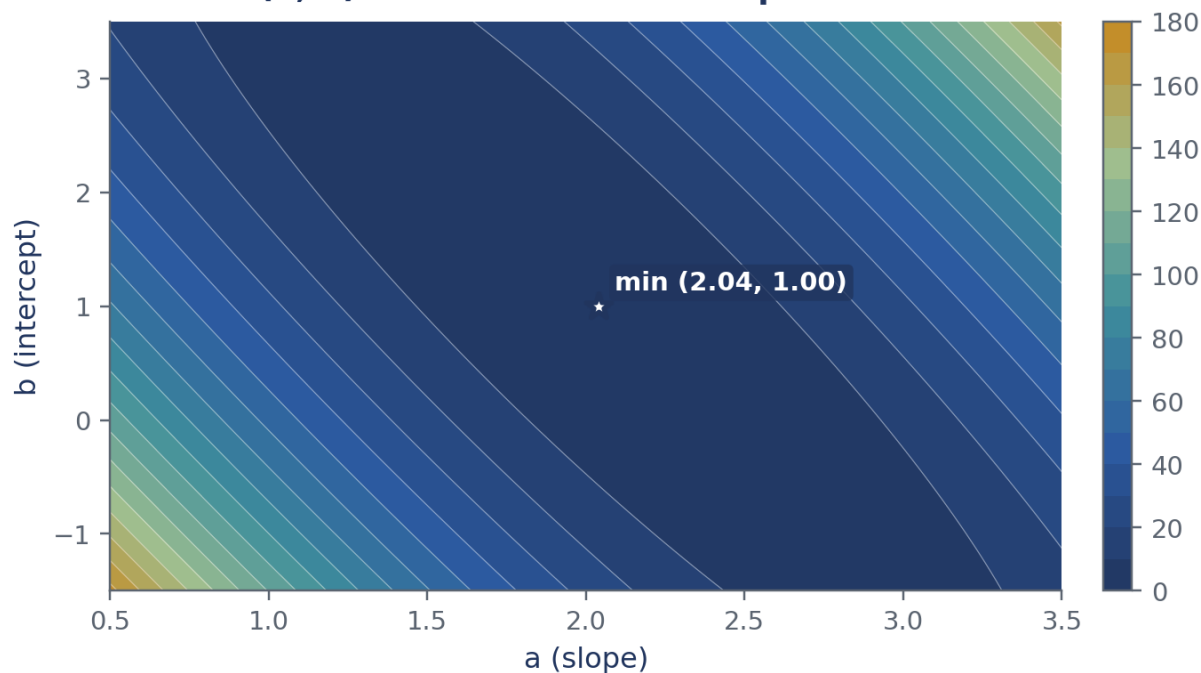


Figure 2: The loss  $L(a, b)$  over the two knobs. How to read it: dark navy is low, warm amber is high; each ring is a constant loss level; the star at  $(2.04, 1.00)$  is the single bottom of this bowl.

**Where this shows up in ML.** This section *is* machine learning in miniature. The line is the model,  $(a, b)$  are its **parameters**, and  $L$  is the training loss. Linear regression, logistic regression, and deep networks all repeat this recipe; only the model and the loss get bigger.

✗ **Watch out**

$L$  is a function of the *parameters*  $(a, b)$ , not of the data  $x$ . The data are fixed numbers baked into the landscape. Also, we square the gaps; we do not just add them. Raw gaps cancel in pairs, so a terrible line could score a perfect zero.

✓ **Key takeaway**

Fitting a model means minimising a loss  $L(\text{parameters})$ . For our line,  $L(a, b) = \sum_i (y_i - (ax_i + b))^2$ , and the best pair  $(2.04, 1.0)$  sits where both partial slopes are zero.

## 2 Reading the landscape: gradients, flat spots, and the second-derivative test

Setting slopes to zero worked because our loss was simple. Before the hard cases, we need the vocabulary of landscapes. Which way is downhill? What does “flat” mean? And how do you tell a valley from a hilltop?

### The intuition

The gradient at a point is an arrow pointing in the steepest uphill direction. Walk against the arrow and you lose height fastest. Where the arrow shrinks to nothing, the ground is flat. You are at a valley floor, a hilltop, or a mountain-pass point.

### ★ Everyday picture

You are on a hilly golf course in thick fog, feeling the ground with your feet. The tilt under your shoes is the gradient: it has a direction (steepest uphill) and a strength (how steep). To descend, step against the tilt. When the ground feels level in every direction, stop. But level ground alone cannot say whether you are in a dip or on a crest.

Now the same story in symbols. For a function  $f$  of one variable  $x$ , the **gradient** is the derivative  $\frac{df}{dx}$ : the slope at the current point. A point where the gradient is zero is a **stationary point**. The slides’ rule for moving: step in the direction of the *negative* gradient, because the gradient itself points toward the greatest increase. Keep stepping until you meet a stationary point. That valley is a **local minimum**: lowest among its neighbours. It may not be the lowest valley anywhere — that one is the **global minimum**.

How do we tell what kind of flat spot we found? Use the **second derivative**  $\frac{d^2f}{dx^2}$ , the slope of the slope. It measures how the ground curves. Positive curvature means the surface bends upward like a cup: a local minimum. Negative curvature bends downward like a dome: a local maximum. This is the **second-derivative test** from the slide.

### Worked example: the slide’s quartic $l(x) = x^4 + 7x^3 + 5x^2 - 17x + 3$ , fully mapped

We find every stationary point and classify each one.

**Step 1. Differentiate.** Power rule, term by term:  $\frac{dl}{dx} = 4x^3 + 21x^2 + 10x - 17$ .

**Step 2. Locate sign changes of the slope.** Evaluate the cubic:  $l'(-5) = -42$ ,  $l'(-4) = +23$ ,  $l'(-2) = +15$ ,  $l'(-1) = -10$ ,  $l'(0) = -17$ ,  $l'(1) = +18$ . The sign flips three times, so there are three roots: one in  $(-5, -4)$ , one in  $(-2, -1)$ , one in  $(0, 1)$ . Three is the most a cubic can have, which matches the slide.

**Step 3. Pin the roots down.** Halving each bracket (bisection) until it is tight gives  $x \approx -4.480$ ,  $-1.432$ , and  $0.662$ .

**Step 4. Differentiate again.**  $\frac{d^2l}{dx^2} = 12x^2 + 42x + 10$ .

**Step 5. Test each point.** At  $x = -4.480$ :  $12(20.07) + 42(-4.480) + 10 = 240.9 - 188.2 + 10 = +62.7$ , positive, so a **local minimum**. At  $x = -1.432$ :  $12(2.051) - 60.1 + 10 = -25.5$ , negative, so a **local maximum**. At  $x = 0.662$ :  $12(0.439) + 27.8 + 10 = +43.1$ , positive, so a **local minimum**.

**Step 6. Compare the two valleys.**  $l(-4.480) = -47.08$  and  $l(0.662) = -3.84$ . The

left valley is far deeper: it is the global minimum, and the right one is only local.

**Step 7. Sense-check.** A quartic with a positive leading term rises at both ends, so “min–max–min” is exactly the shape it must have.

One function, two valleys of different depths: this is why “a” minimum and “the” minimum are different promises.

### $l(x)$ : two valleys and one hilltop between them

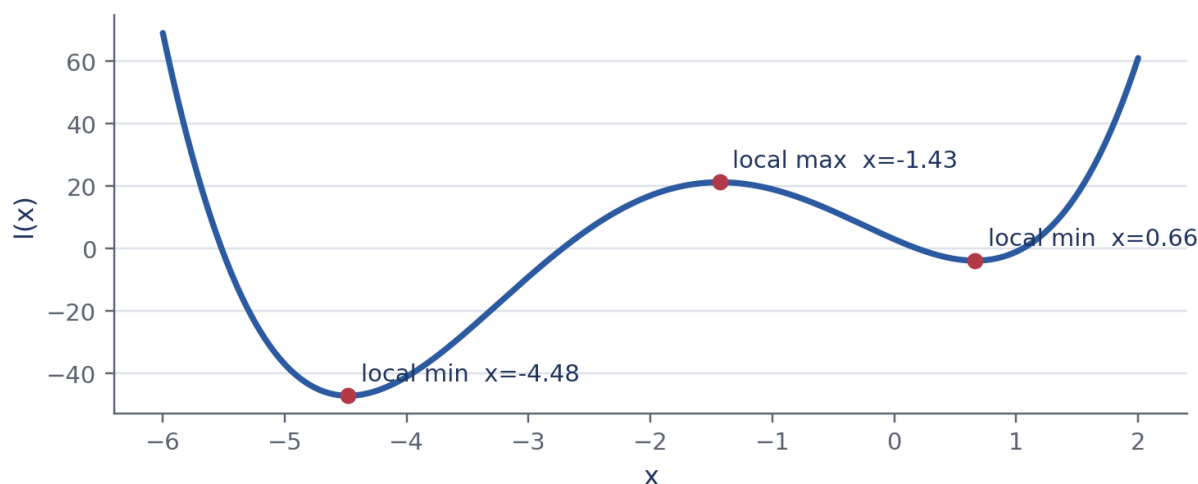


Figure 3: The quartic from the slides. How to read it: the three dots are the stationary points where  $dl/dx = 0$ ; curvature (the second derivative) separates the two valleys from the hilltop between them, and the left valley is the deeper, global one.

**Where this shows up in ML.** A deep network’s loss surface is exactly this picture in millions of dimensions: many valleys of different depths. Training finds *a* valley, not *the* valley, and where you start decides which one you roll into. In many dimensions the second derivative becomes a matrix of curvatures called the **Hessian**; its signs classify flat points the same way.

#### ⚠ Watch out

“Gradient equals zero” does not mean “minimum”. It could be a maximum, or (in higher dimensions) a saddle: a mountain pass that curves up one way and down another. Always ask the curvature before celebrating.

#### ✓ Key takeaway

Downhill is the negative gradient. Stationary means gradient zero. Positive second derivative marks a local minimum, negative marks a local maximum, and the deepest valley of all is the global minimum.

### 3 Gradient descent: the walk-downhill algorithm

For a low-order polynomial we just solved “gradient = 0” exactly. Real models are not so kind. We want the minimum of a function  $f : \mathbb{R}^d \rightarrow \mathbb{R}$ :  $d$  inputs in, one score out. Here  $d$  can be in the millions, and no closed-form solution exists. We assume only that  $f$  is **differentiable**: smooth enough to have a gradient everywhere. The plan: stop trying to *solve* for the bottom and *walk* there instead.

#### The intuition

Gradient descent is “feel the slope, step downhill, repeat”. It never sees the whole landscape. It only ever uses the tilt right under its feet, which is cheap to compute even when the landscape is enormous.

The update rule from the slide, one careful piece at a time. Start from a guess  $\mathbf{x}_0$ . Then repeat:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \gamma_i ((\nabla f)(\mathbf{x}_i))^{\top}. \quad (4)$$

Read it symbol by symbol.  $\mathbf{x}_i$  is our position after  $i$  steps.  $\nabla f$  (read “nabla f” or “grad f”) is the gradient: the vector of all partial slopes. And  $\gamma_i$  (the Greek letter gamma) is the **step size** at step  $i$ . Machine learning calls it the **learning rate**. The little  $\top$  is a **transpose**. The lecture’s convention writes  $\nabla f$  as a row of numbers. The transpose stands that row up into a column, so it can be subtracted from the column  $\mathbf{x}_i$ . The minus sign is the entire trick: the gradient points uphill, so we subtract it to go downhill.

Why does this work? For a small enough  $\gamma > 0$ , one step is guaranteed not to hurt:  $f(\mathbf{x}_1) \leq f(\mathbf{x}_0)$ . Chain such steps and the scores form a falling staircase,  $f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \dots$ , which (for suitable step sizes  $\gamma_i$ ) settles into a local minimum. Gradient descent is called a **first-order** method because it uses only first derivatives — slopes, never curvatures.

#### Worked example: a warm-up run on $f(x) = x^2$

One knob, start at  $x_0 = 4$ , fixed step size  $\gamma = 0.3$ . The gradient is  $f'(x) = 2x$ .

**Step 1.** Slope at 4:  $f'(4) = 8$ . Update:  $x_1 = 4 - 0.3 \times 8 = \mathbf{1.6}$ . Loss:  $16 \rightarrow 2.56$ .

**Step 2.** Slope:  $f'(1.6) = 3.2$ . Update:  $x_2 = 1.6 - 0.96 = \mathbf{0.64}$ . Loss:  $2.56 \rightarrow 0.4096$ .

**Step 3.** Slope: 1.28. Update:  $x_3 = 0.64 - 0.384 = \mathbf{0.256}$ . Loss:  $0.4096 \rightarrow 0.0655$ .

**Step 4. Spot the pattern.** Each update is  $x - 0.3(2x) = (1 - 0.6)x = 0.4x$ . Every step multiplies the distance to the minimum by 0.4, so the walk homes in geometrically.

**Step 5. Sense-check.** The loss fell every step, as the small-step guarantee promises, and  $x$  is marching toward the true minimum 0.

A falling staircase of losses, powered only by local slopes.

**Where this shows up in ML.** This loop *is* the training loop. Frameworks like PyTorch compute  $\nabla f$  for every weight at once (that is what backpropagation does), then apply exactly this update. “First-order only” is why it scales: slopes for a billion weights are affordable, curvatures are not.

## The quadratic bowl in 3-D — the ball rolls to the floor

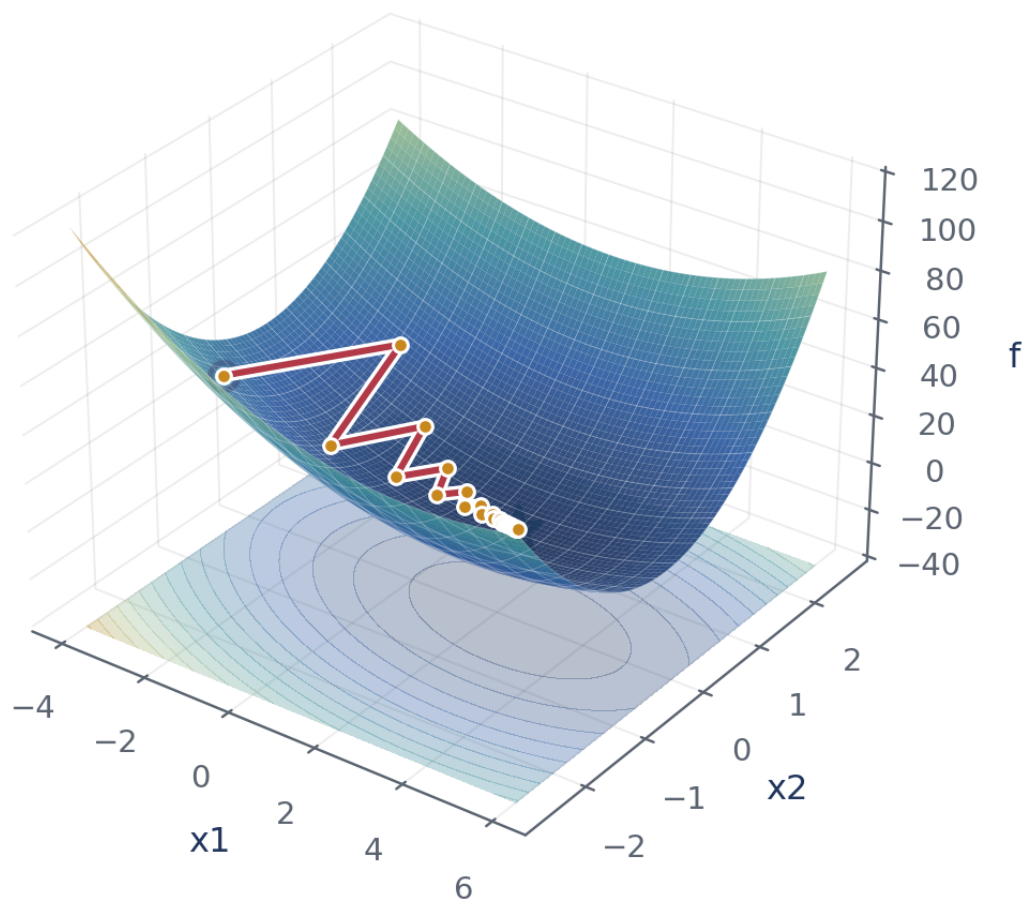


Figure 4: The two-knob bowl we descend in the next section, drawn in 3-D. How to read it: height is the loss  $f$ ; the amber trail is a gradient-descent run rolling from the rim to the floor; the shadow below repeats the path on a flat contour map.



**⚠ Watch out**

The guarantee is local, in two senses. The step must be small enough, or the loss can rise (next sections). And the destination is only a *local* minimum: whichever valley your start point drains into.

**✓ Key takeaway**

Gradient descent repeats  $\mathbf{x}_{i+1} = \mathbf{x}_i - \gamma_i (\nabla f(\mathbf{x}_i))^\top$ : step against the gradient, scaled by the step size, until the gradient dies out at a local minimum.

## 4 Worked in full: descending a quadratic bowl

The slides now run gradient descent on a concrete two-knob function.

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x} + \mathbf{b}^\top \mathbf{x}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad A = \begin{bmatrix} 2 & 1 \\ 1 & 20 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -5 \\ -3 \end{bmatrix}. \quad (5)$$

Unpack it once so nothing is mysterious. The quadratic part multiplies out to

$$\frac{1}{2} \mathbf{x}^\top A \mathbf{x} = \frac{1}{2} (2x_1^2 + 2x_1x_2 + 20x_2^2) = x_1^2 + x_1x_2 + 10x_2^2, \quad (6)$$

and the linear part is  $\mathbf{b}^\top \mathbf{x} = -5x_1 - 3x_2$ . So  $f$  is an ordinary bowl-shaped polynomial:  $f = x_1^2 + x_1x_2 + 10x_2^2 - 5x_1 - 3x_2$ . From the vector-calculus lecture, its gradient (as a row) is  $\nabla f = \mathbf{x}^\top A + \mathbf{b}^\top$ . Because  $A$  is symmetric, that row is just  $(A\mathbf{x} + \mathbf{b})^\top$ , with components

$$\nabla f = (2x_1 + x_2 - 5, \quad x_1 + 20x_2 - 3). \quad (7)$$

### Worked example: where is the true bottom of this bowl?

The floor is where the gradient vanishes:  $A\mathbf{x} = -\mathbf{b}$ .

**Step 1. Write the two flat-ground equations.**  $2x_1 + x_2 = 5$  and  $x_1 + 20x_2 = 3$ .

**Step 2. Solve by substitution.** From the first,  $x_2 = 5 - 2x_1$ . Put that into the second:  $x_1 + 100 - 40x_1 = 3$ , so  $-39x_1 = -97$  and  $x_1 = 97/39 = \mathbf{2.4872}$ .

**Step 3. Back-substitute.**  $x_2 = 5 - 2(2.4872) = \mathbf{0.0256}$ .

**Step 4. Value at the bottom.** At the minimum  $A\mathbf{x}_* = -\mathbf{b}$ , so  $f(\mathbf{x}_*) = \frac{1}{2} \mathbf{x}_*^\top (-\mathbf{b}) + \mathbf{b}^\top \mathbf{x}_* = \frac{1}{2} \mathbf{b}^\top \mathbf{x}_*$ . Numerically:  $\frac{1}{2} [(-5)(2.4872) + (-3)(0.0256)] = \frac{1}{2} (-12.5128) = \mathbf{-6.2564}$ .

**Step 5. Sense-check.** Feed  $(2.4872, 0.0256)$  into the gradient:  $2(2.4872) + 0.0256 - 5 = 0.0000$  and  $2.4872 + 20(0.0256) - 3 = -0.0008$ , zero up to rounding. The ground is flat there.

Now we know the target, so we can watch gradient descent find it.

The slide starts the walk at  $\mathbf{x}_0 = (-3, -1)$  and iterates the update rule (its MATLAB code is `grad_descent1.m`). We take  $\gamma = 0.085$  and do the first two steps by hand.

### Worked example: two gradient-descent steps from $(-3, -1)$ with $\gamma = 0.085$

**Step 1. Loss at the start.**  $f(-3, -1) = 9 + 3 + 10 + 15 + 3 = \mathbf{40}$ .

**Step 2. Gradient at the start.**  $(2(-3) + (-1) - 5, -3 + 20(-1) - 3) = \mathbf{(-12, -26)}$ .

**Step 3. First update.**  $\mathbf{x}_1 = (-3, -1) - 0.085(-12, -26) = (-3 + 1.02, -1 + 2.21) = \mathbf{(-1.98, 1.21)}$ .

**Step 4. Loss after one step.**  $f(-1.98, 1.21) = 3.9204 - 2.3958 + 14.641 + 9.9 - 3.63 = \mathbf{22.44}$ . Down from 40.

**Step 5. Gradient there.**  $(2(-1.98) + 1.21 - 5, -1.98 + 20(1.21) - 3) = \mathbf{(-7.75, 19.22)}$ .

**Step 6. Second update.**  $\mathbf{x}_2 = (-1.98, 1.21) - 0.085(-7.75, 19.22) =$

$(-1.3213, -0.4237)$ .

**Step 7. Loss again.**  $f(x_2) = 11.98$ ; a third step gives 5.58. The staircase  $40 \rightarrow 22.44 \rightarrow 11.98 \rightarrow 5.58$  is falling toward the floor value  $-6.2564$ .

**Step 8. Sense-check.** Each loss beats the last, as the small-step guarantee promises, and the points drift toward  $(2.4872, 0.0256)$ .

Note the swing in  $x_2$ :  $-1 \rightarrow 1.21 \rightarrow -0.42$ . The valley is much steeper in the  $x_2$  direction (the 20 in  $A$ ), so the walker overshoots across it while crawling along  $x_1$ . Keep this zig-zag in mind for the step-size sections.

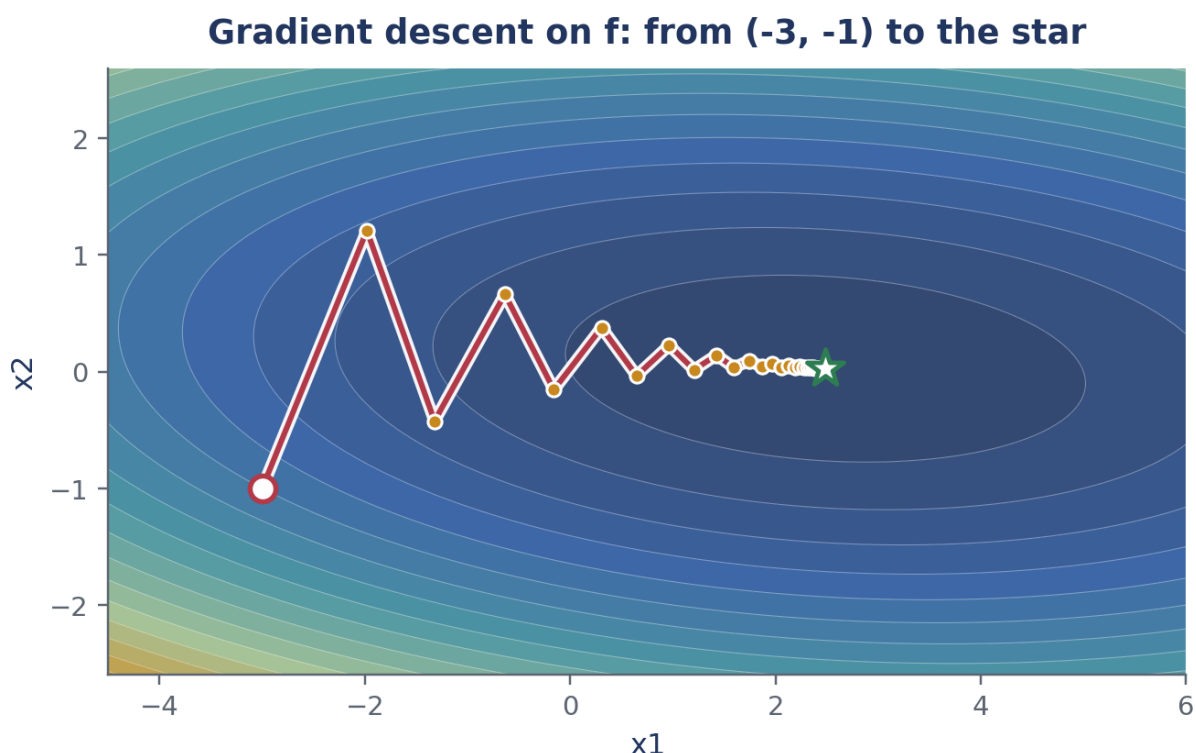


Figure 5: Thirty steps of the run above. How to read it: rings are constant-loss levels of the long, narrow bowl; the trail starts at the ring marker  $(-3, -1)$ , bounces across the steep direction, and lands on the star at  $(2.4872, 0.0256)$ .

**Where this shows up in ML.** Near any smooth minimum, every loss looks like this quadratic bowl — that is what a second-order Taylor expansion says. Long narrow bowls (here: curvature 20 versus 2) are called **ill-conditioned**, and they are why plain gradient descent zig-zags on real models. Momentum methods and Adam exist largely to tame this shape.

#### ⚠ Watch out

The step size that worked here is fragile. For this bowl, any fixed  $\gamma$  above about 0.0997 makes the steps grow instead of shrink — the walk explodes. That ceiling is set by the steepest curvature in  $A$  (its largest eigenvalue, 20.06), a preview of the next section.

**✓ Key takeaway**

On this quadratic bowl, gradient descent with a sane step size walks from  $(-3, -1)$  down to the exact floor  $(2.4872, 0.0256)$ . That is the same point “solve  $A\mathbf{x} = -\mathbf{b}$ ” gives in one line.

## 5 The step size: too small crawls, too big explodes

Everything in gradient descent hangs on one number,  $\gamma$ . The slide states the dilemma. Too small, and the walk becomes painfully slow. Too large, and the steps overshoot the target, fail to converge, or even diverge — each stride lands *higher* than the last.

### The intuition

The gradient only tells you the tilt where you stand, not how far the valley floor is. The step size is your leap of faith. Small leaps trust the slope for a millimetre; huge leaps trust it for a mile it never promised.

### ★ Everyday picture

Walking downstairs in the dark, you can shuffle one toe-length at a time and take all night. Or you can jump three steps at once and crack your shin on the far wall. The stairs did not change; your stride did. The analogy breaks in one place: stairs forgive a big jump once, while a diverging optimiser gets *worse* every jump.

### Worked example: one bowl, three fates: $f(x) = x^2$ from $x_0 = 4$

The update is  $x \leftarrow x - \gamma(2x) = (1 - 2\gamma)x$ : each step just multiplies  $x$  by the factor  $(1 - 2\gamma)$ .

**Step 1. Small step,**  $\gamma = 0.05$ . Factor = 0.9. Iterates:  $4 \rightarrow 3.6 \rightarrow 3.24 \rightarrow 2.916 \rightarrow \dots$   
After ten steps  $x$  is still 1.39. It converges, at a crawl.

**Step 2. Healthy step,**  $\gamma = 0.45$ . Factor = 0.1. Iterates:  $4 \rightarrow 0.4 \rightarrow 0.04 \rightarrow \mathbf{0.004}$ .  
Three steps do what the crawl could not do in ten.

**Step 3. Reckless step,**  $\gamma = 1.05$ . Factor =  $-1.1$ . Iterates:  $4 \rightarrow -4.4 \rightarrow 4.84 \rightarrow -5.324$ .  
The sign flips each time (it jumps across the valley) and the size *grows* 10% per step: **divergence**.

**Step 4. Read off the safety rule.** The walk shrinks exactly when  $|1 - 2\gamma| < 1$ , which solves to  $0 < \gamma < 1$ . The knife edge  $\gamma = 1$  bounces between  $\pm 4$  forever.

**Step 5. Sense-check.** At  $\gamma = 0.5$  the factor is 0: one perfect step to the bottom. Both neighbours of that sweet spot behave as the table predicts.

Same landscape, same start, three different stories — told entirely by  $\gamma$ .

The slide then offers a practical fix: make the step size *adaptive*. Try a step. If the loss went *up*, you were too aggressive: undo the step and halve the step size. If the loss went *down*, keep the result and let the next step be a bit larger. Either way, the rule only ever accepts moves that lower the loss. So the recorded loss never rises. That is what “guarantees monotonic convergence” means on the slide.

### Worked example: the undo-and-halve rule rescuing a bad start

Take  $f(x) = x^2$ ,  $x_0 = 4$ , and a deliberately bad first guess  $\gamma = 1.2$ .

**Step 1. Try the step.** Slope = 8, so the proposal is  $x = 4 - 1.2(8) = -5.6$ . Loss check:  $f(-5.6) = 31.36 > 16$ . Worse!

**Step 2. Undo and halve.** Return to  $x = 4$ ; set  $\gamma = 0.6$ .

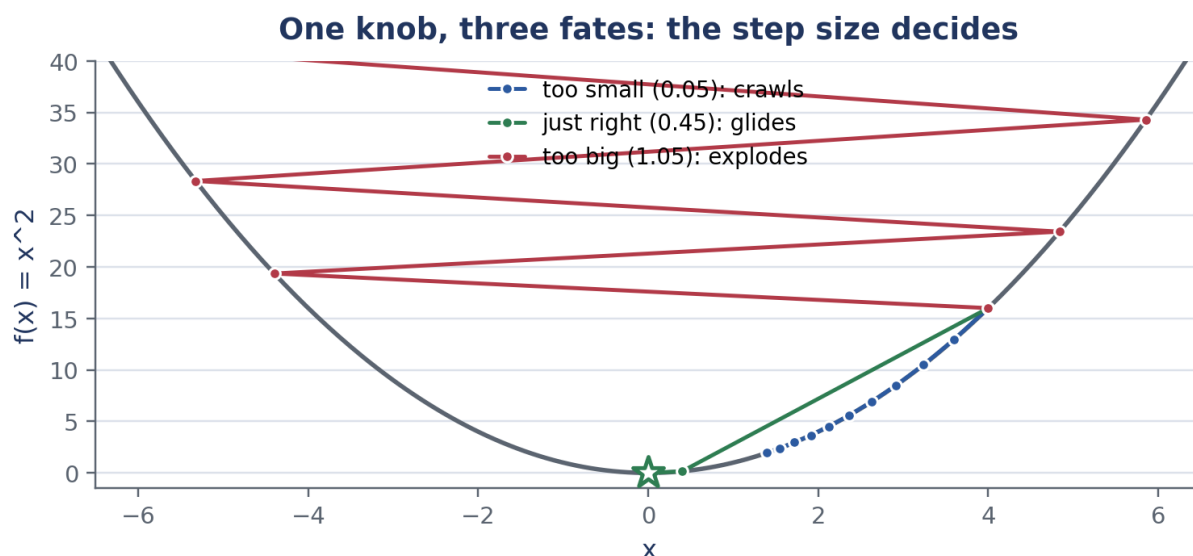


Figure 6: The three runs above, drawn on the bowl. How to read it: blue dots crowd the rim (crawl), green dots glide to the star (healthy), red dots leap the valley with growing height (diverge).

**Step 3. Try again.** Proposal:  $x = 4 - 0.6(8) = -0.8$ . Loss check:  $f(-0.8) = \mathbf{0.64} < 16$ . Accept.

**Step 4. Continue.** From  $-0.8$  the same  $\gamma$  gives  $x = -0.8 + 0.96 = 0.16$ , loss 0.0256. Falling nicely.

**Step 5. Sense-check.** The recorded losses ran  $16 \rightarrow 16 \rightarrow 0.64 \rightarrow 0.0256$ : never up. The one bad proposal cost a try, not progress.

The optimiser learned its own step size from two probes.

**Where this shows up in ML.** The learning rate is the single most tuned hyperparameter in deep learning. Divergence has a famous signature: the loss becomes NaN. Undo-and-halve survives inside modern tricks like “reduce learning rate on plateau” and gradient-clipping safety nets.

#### ⚠ Watch out

“Monotonic” promises the loss never rises; it does not promise speed. Halving too eagerly can shrink  $\gamma$  to almost nothing and stall the walk. And a  $\gamma$  that is safe in one region can be reckless in a steeper one.

#### ✓ Key takeaway

The step size trades speed against stability: crawl, glide, or explode. The undo-and-halve heuristic buys safety — every accepted step lowers the loss — at the price of occasional wasted probes.

## 6 The best possible step: exact line search on a quadratic

Why guess  $\gamma$  at all? Once we fix the downhill direction, the loss along that direction is a one-variable function of  $\gamma$ . One variable means we can sometimes minimise it exactly, with calculus. The slide does precisely this for  $f(x, y) = x^2 + 3y^2$ , starting at  $(2, 2)$ ; its code is `grad_descent2.m`.

### The intuition

Freeze the direction, then slide along it like a bead on a wire. The loss on the wire is a plain curve in  $\gamma$ . Its lowest point is the best possible step in that direction — no guessing, just a one-variable minimisation.

Build the one-variable curve, symbol by symbol. Define  $h(\gamma) = f(\mathbf{x}_{i+1}) = f(\mathbf{x}_i - \gamma(\nabla f(\mathbf{x}_i))^\top)$ : the loss after a step of size  $\gamma$ . For our  $f$  the gradient is  $\nabla f(x, y) = [2x, 6y]$ , so one step moves the point like this.

$$x \rightarrow x - \gamma(2x) = (1 - 2\gamma)x, \quad y \rightarrow y - \gamma(6y) = (1 - 6\gamma)y. \quad (8)$$

Substitute the moved point back into  $f$ .

$$h(\gamma) = (1 - 2\gamma)^2 x^2 + 3(1 - 6\gamma)^2 y^2. \quad (9)$$

Now minimise  $h$  the section-1 way: differentiate and set to zero. Chain rule on each term (outer square down, times inner slope  $-2$  or  $-6$ ):

$$h'(\gamma) = -4(1 - 2\gamma)x^2 - 36(1 - 6\gamma)y^2 = 0. \quad (10)$$

Expand and collect the  $\gamma$  terms:  $-4x^2 + 8\gamma x^2 - 36y^2 + 216\gamma y^2 = 0$ , so  $\gamma(8x^2 + 216y^2) = 4x^2 + 36y^2$ . Divide both sides by 4:

$$\gamma = \frac{x^2 + 9y^2}{2x^2 + 54y^2} \quad (11)$$

which is exactly the slide's formula: the best step size, recomputed fresh at every point.

### Worked example: two exact-step moves from $(2, 2)$

**Step 1. Best step at  $(2, 2)$ .** Here  $x^2 = y^2 = 4$ :  $\gamma = \frac{4 + 36}{8 + 216} = \frac{40}{224} = \frac{5}{28} \approx \mathbf{0.1786}$ .

**Step 2. Move.**  $x_1 = (1 - 2\gamma)2 = (0.6429)(2) = \mathbf{1.2857}$ ;  $y_1 = (1 - 6\gamma)2 = (-0.0714)(2) = \mathbf{-0.1429}$ .

**Step 3. Loss drop.**  $f(2, 2) = 4 + 12 = 16$ , and  $f(1.2857, -0.1429) = 1.6531 + 0.0612 = \mathbf{1.7143}$ . One step removed 89% of the loss.

**Step 4. Best step at the new point.** Now  $x^2 = 1.6531$ ,  $y^2 = 0.0204$ :  $\gamma = \frac{1.6531 + 0.1837}{3.3061 + 1.1020} = \frac{1.8367}{4.4082} = \frac{5}{12} \approx \mathbf{0.4167}$ . The steep direction is nearly spent, so a bolder step is optimal.

**Step 5. Move again.**  $x_2 = (1 - 0.8333)(1.2857) = \mathbf{0.2143}$ ;  $y_2 = (1 - 2.5)(-0.1429) =$

**0.2143.** Loss:  $f = 0.0459 + 0.1378 = \mathbf{0.1837}$ .

**Step 6. Sense-check.** Losses  $16 \rightarrow 1.714 \rightarrow 0.184$  collapse fast, and the two step sizes repeat  $(5/28, 5/12, 5/28, \dots)$  as this bowl's geometry dictates.

The slide's closing caution stands: recomputing the perfect  $\gamma$  each step can be expensive. Other methods are often used instead — that trade is Sections 10–13.

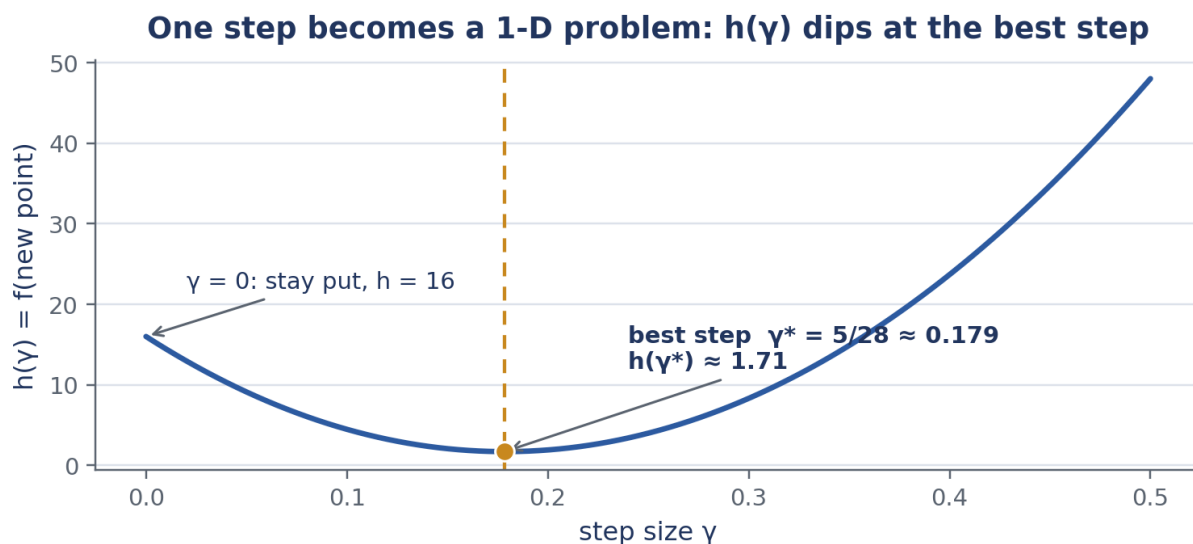


Figure 7: The one-variable curve  $h(\gamma)$  for the first step above. How to read it: sliding from  $\gamma = 0$  (stay put, loss 16) the curve dips to its floor at  $\gamma^* = 5/28$  — the best step — and rises again when the step overshoots.

**Where this shows up in ML.** This “bead on a wire” picture seeds every line-search method in classical optimisation. It also powers the learning-rate finders in modern libraries. They probe a range of  $\gamma$  values before training starts.

#### ✗ Watch out

The tidy closed-form  $\gamma$  exists because  $f$  was quadratic. For a general loss,  $h(\gamma)$  has no such formula. Each probe of  $h$  then costs a full model evaluation — the expense the slide warns about.

#### ✓ Key takeaway

Fixing the direction turns step-size choice into minimising a one-variable function  $h(\gamma)$ . On  $x^2 + 3y^2$  the answer is exact:  $\gamma = (x^2 + 9y^2)/(2x^2 + 54y^2)$ , recomputed at every step.



## 7 Is my gradient code right? The finite-difference check

Gradient descent trusts the gradient completely. If your derivative code has a bug, the walk marches confidently in a wrong direction. The slide gives the standard safety net: check the analytic gradient against a purely numerical estimate that needs no calculus at all.

### The intuition

A derivative is “rise over run” in the limit of a tiny run. So fake the limit: nudge one input by a small amount, watch how much the output moves, and divide. If that ratio disagrees with your formula, your formula is wrong.

### ★ Everyday picture

To check a car’s speedometer, you do not open the engine. You time yourself between two mile markers and divide distance by time. The finite-difference check is the mile-marker test for your gradient code. Slower than the speedometer, but nearly impossible to get wrong.

The formula, unpacked. Let  $J(\mathbf{w}) = J(w_1, w_2, \dots, w_d)$  be the loss over  $d$  parameters. Pick one parameter  $w_i$ , nudge it by a small  $\Delta$  (capital delta), and leave the rest alone.

$$\frac{\partial J}{\partial w_i} \approx \frac{J(w_1, \dots, w_i + \Delta, \dots, w_d) - J(w_1, \dots, w_i, \dots, w_d)}{\Delta}. \quad (12)$$

The top is “how much  $J$  rose”; the bottom is “how far we pushed”. Because computing  $J$  twice per parameter is slow, the slide says to *sample* a few parameters  $w_i$  and spot-check those, not all  $d$ .

### Worked example: checking both slopes of $J(w_1, w_2) = w_1^2 + 3w_1w_2$ at $(1, 2)$

**Step 1. Analytic slopes first.** Differentiating:  $\frac{\partial J}{\partial w_1} = 2w_1 + 3w_2$  and  $\frac{\partial J}{\partial w_2} = 3w_1$ . At  $(1, 2)$  they claim  $2 + 6 = 8$  and  $3$ .

**Step 2. Base value.**  $J(1, 2) = 1 + 6 = 7$ .

**Step 3. Nudge  $w_1$  by  $\Delta = 0.01$ .**  $J(1.01, 2) = 1.0201 + 6.06 = 7.0801$ . Estimate:  $(7.0801 - 7)/0.01 = 8.0100$ . Against the claimed 8: off by 0.01.

**Step 4. Why exactly 0.01?** The estimate leans on the curve ahead of the point, so it picks up half the curvature times  $\Delta$ : here  $\frac{1}{2}(2)(0.01) = 0.01$ . The mismatch is the method, not a bug.

**Step 5. Nudge  $w_2$  by  $\Delta = 0.01$ .**  $J(1, 2.01) = 1 + 6.03 = 7.03$ . Estimate:  $(7.03 - 7)/0.01 = 3.0000$ , agreeing perfectly —  $J$  is a straight line in  $w_2$ , and straight lines have no curvature error.

**Step 6. Sense-check.** Both estimates match the calculus to the expected accuracy, so the gradient code passes.

Two cheap nudges just certified our calculus.

**Where this shows up in ML.** This is **gradient checking**, the classic unit test for hand-written backpropagation layers. Frameworks automate it (for example `torch.autograd.gradcheck`), comparing autograd against finite differences on a few sampled coordinates — exactly the slide’s

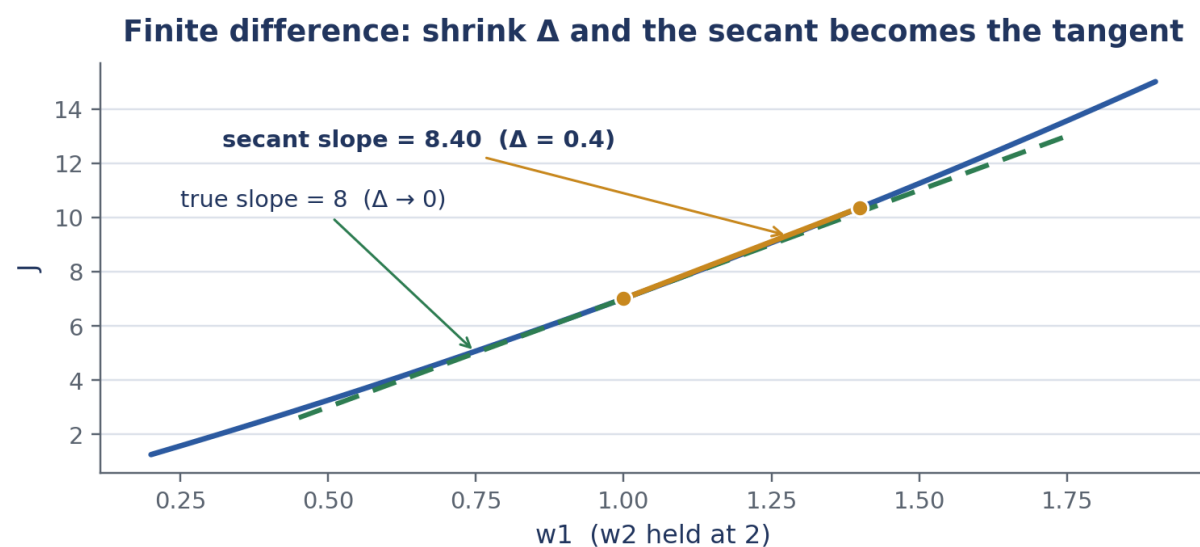


Figure 8: The check as a picture (slice along  $w_1$  with  $\Delta$  exaggerated to 0.4 so the gap is visible). How to read it: the amber secant through two nearby points tilts at 8.40; as  $\Delta$  shrinks it rotates onto the green tangent of slope exactly 8.

advice.

#### ✗ Watch out

$\Delta$  is a compromise. Too large, and curvature pollutes the estimate. Too small, and floating-point rounding eats the numerator — subtracting two nearly equal numbers destroys precision. Around  $10^{-5}$  works well, and the two-sided version  $(J(w_i + \Delta) - J(w_i - \Delta)) / (2\Delta)$  cancels the curvature term for extra accuracy.

#### ✓ Key takeaway

Estimate each slope as (change in loss)/(size of nudge) and compare with your formula. Agreement to a few decimal places means the gradient code is safe to trust.

## 8 Learning-rate decay: start bold, finish careful

Back to choosing the rate, now with a time dimension. The slides ask: what if the learning rate is *constant*? A large constant races toward the target almost immediately. Then it *oscillates* around the target forever, each step leaping across the minimum to the far side. A small constant avoids the dance but takes a very long time to arrive. Neither behaviour is what we want.

### The intuition

You want two different step sizes in one run: big strides while you are far away, tiptoes once you are close. A decaying learning rate gives you both — large first, shrinking with time.

### ★ Everyday picture

Parking a car, you cover the lot at speed, brake as you near the bay, and inch the last half-metre. Drive the whole way at highway speed and you overshoot the bay endlessly. Drive the whole way at parking speed and you never arrive. Decay is easing off the accelerator as you close in.

With a time-dependent rate the update becomes  $\mathbf{w}_{n+1} = \mathbf{w}_n - \alpha_t \nabla J$ , where  $\mathbf{w}_n$  holds the parameters after  $n$  updates and  $\alpha_t$  (alpha) is the rate at time  $t$ . The slide measures  $t$  in **epochs**: full cycles through the training data, so the rate stays constant within one cycle. The two most common decay schedules are

$$\text{exponential decay: } \alpha_t = \alpha_0 e^{-kt}, \quad (13)$$

$$\text{inverse decay: } \alpha_t = \frac{\alpha_0}{1 + kt}, \quad (14)$$

where  $\alpha_0$  is the starting rate and  $k$  controls how fast the shrinking happens. A third pattern, **step decay**, simply cuts the rate by a fixed factor every few epochs — a staircase rather than a slide.

### Worked example: one table, both formulas: $\alpha_0 = 0.5$ , $k = 0.5$

We compute the first five epochs of each schedule.

**Step 1. Exponential at  $t = 1$ .**  $\alpha_1 = 0.5 e^{-0.5} = 0.5 \times 0.6065 = \mathbf{0.3033}$ .

**Step 2. Inverse at  $t = 1$ .**  $\alpha_1 = \frac{0.5}{1 + 0.5} = \frac{0.5}{1.5} = \mathbf{0.3333}$ .

**Step 3. Fill the table the same way.**

| $t$                    | 0   | 1      | 2      | 3      | 4      |
|------------------------|-----|--------|--------|--------|--------|
| exponential $\alpha_t$ | 0.5 | 0.3033 | 0.1839 | 0.1116 | 0.0677 |
| inverse $\alpha_t$     | 0.5 | 0.3333 | 0.2500 | 0.2000 | 0.1667 |

**Step 4. Read the difference.** By epoch 4 the exponential rate has lost 86% of its size; the inverse rate only 67%. Exponential decay brakes hard; inverse decay keeps a long, useful tail.

**Step 5. Sense-check.** Both start at exactly  $\alpha_0$  at  $t = 0$ , because  $e^0 = 1$  and  $1 + 0 = 1$ . Both shrink toward zero as  $t$  grows — big strides first, tiptoes later, as designed.

Same idea, two personalities: choose by how long you want boldness to last.

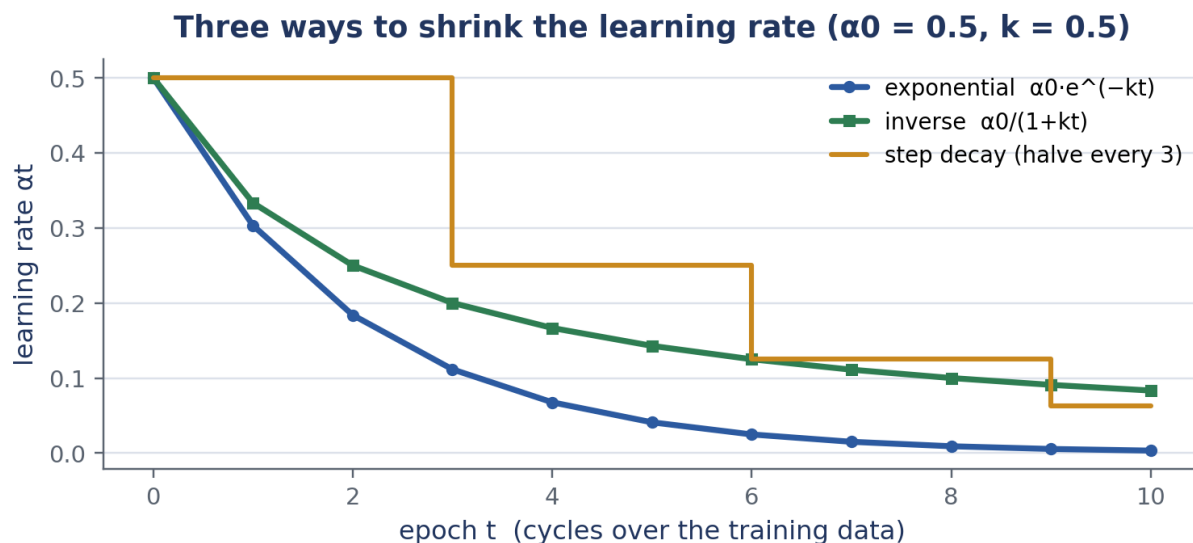


Figure 9: The three schedules over ten epochs. How to read it: all start at  $\alpha_0 = 0.5$ ; the blue exponential dives fastest, the green inverse keeps a heavier tail, and the amber staircase is step decay halving every three epochs.

**Where this shows up in ML.** Every serious training run schedules its learning rate. Step decay trained the classic ResNets. Exponential and inverse decay are one line in every framework. Modern recipes add warm-up and cosine shapes. The reason is exactly the slide's: constant-large oscillates, constant-small crawls.

#### ⚠ Watch out

Decay that is too aggressive freezes the run. The rate hits “tiptoe” while you are still far from the valley, and progress stops. The starting rate  $\alpha_0$  still matters as much as the schedule — decay cannot rescue a divergent first epoch.

#### ✓ Key takeaway

Replace the constant rate with  $\alpha_t$ : exponential decay  $\alpha_0 e^{-kt}$ , inverse decay  $\alpha_0 / (1 + kt)$ , or a step-decay staircase. Big steps early for speed, small steps late for a clean landing.

## 9 The bold driver: a step size that tunes itself

Decay schedules shrink the rate blindly, on a timer. The **bold-driver** algorithm listens instead. It watches whether the loss is improving or worsening, and it adjusts the learning rate every iteration.

### The intuition

Drive bolder while the road rewards you; the moment you crash, back up and drive at half speed. Improvement earns a slightly bigger rate. A worsening step is undone — not kept — and the rate is cut hard.

The slide’s rules, exactly: while steps improve the objective, grow the learning rate by about 5% per iteration ( $\alpha \leftarrow 1.05 \alpha$ ). The moment a step makes the objective *worse*, undo that step and cut the rate by 50% ( $\alpha \leftarrow 0.5 \alpha$ ). Then take a fresh step with the new rate. Continue until convergence.

### Worked example: six bold-driver iterations on $f(x) = x^2$ , from $x_0 = 4$ , $\alpha_0 = 0.9$

The gradient is  $2x$ ; each row tries  $x_{\text{new}} = x - \alpha(2x)$ .

**Step 1. Iteration 1.** Slope 8; try  $x = 4 - 0.9(8) = -3.2$ . Loss  $16 \rightarrow 10.24$ : better. Keep it; grow  $\alpha = 0.9 \times 1.05 = \mathbf{0.945}$ .

**Step 2. Iteration 2.** Slope  $-6.4$ ; try  $-3.2 + 6.048 = 2.848$ . Loss  $10.24 \rightarrow 8.111$ : better. Keep;  $\alpha = \mathbf{0.9923}$ .

**Step 3. Iteration 3.** Slope  $5.696$ ; try  $2.848 - 5.652 = -2.804$ . Loss  $8.111 \rightarrow 7.862$ : better, barely. Keep;  $\alpha = \mathbf{1.0419}$ .

**Step 4. Iteration 4 — the crash.** Slope  $-5.608$ ; try  $-2.804 + 5.843 = 3.039$ . Loss  $7.862 \rightarrow 9.233$ : **worse**. Undo (stay at  $-2.804$ ); halve to  $\alpha = \mathbf{0.5209}$ .

**Step 5. Iteration 5.** Fresh try:  $-2.804 + 2.921 = 0.117$ . Loss  $7.862 \rightarrow \mathbf{0.0138}$ : a huge improvement. Keep;  $\alpha = 0.5470$ .

**Step 6. Iteration 6.** Slope  $0.235$ ; try  $0.117 - 0.128 = -0.011$ . Loss  $\rightarrow \mathbf{0.0001}$ . Essentially arrived.

**Step 7. Sense-check.** The kept losses only ever fell: 16, 10.24, 8.11, 7.86, 0.014, 0.0001. The one bad proposal was discarded, exactly as the rules demand.

Notice the drama. Three greedy raises pushed  $\alpha$  past this bowl’s safe ceiling of 1 (Section 5). The crash was caught, and the halved rate then made spectacular progress.

The slide adds one warning for machine learning. When the objective is estimated from *samples* of data, each measurement is noisy. A single “worse” reading may be sampling luck, not a bad rate. The fix: judge improvement over  $m$  update steps rather than a single step, and adjust the rate on that steadier verdict.

**Where this shows up in ML.** Bold driver is the ancestor of today’s adaptive training tricks. “Reduce learning rate on plateau” schedulers share its reflex. So do trust-ratio methods and per-weight adapters like Adagrad and Adam. Listen to progress, then resize the step.

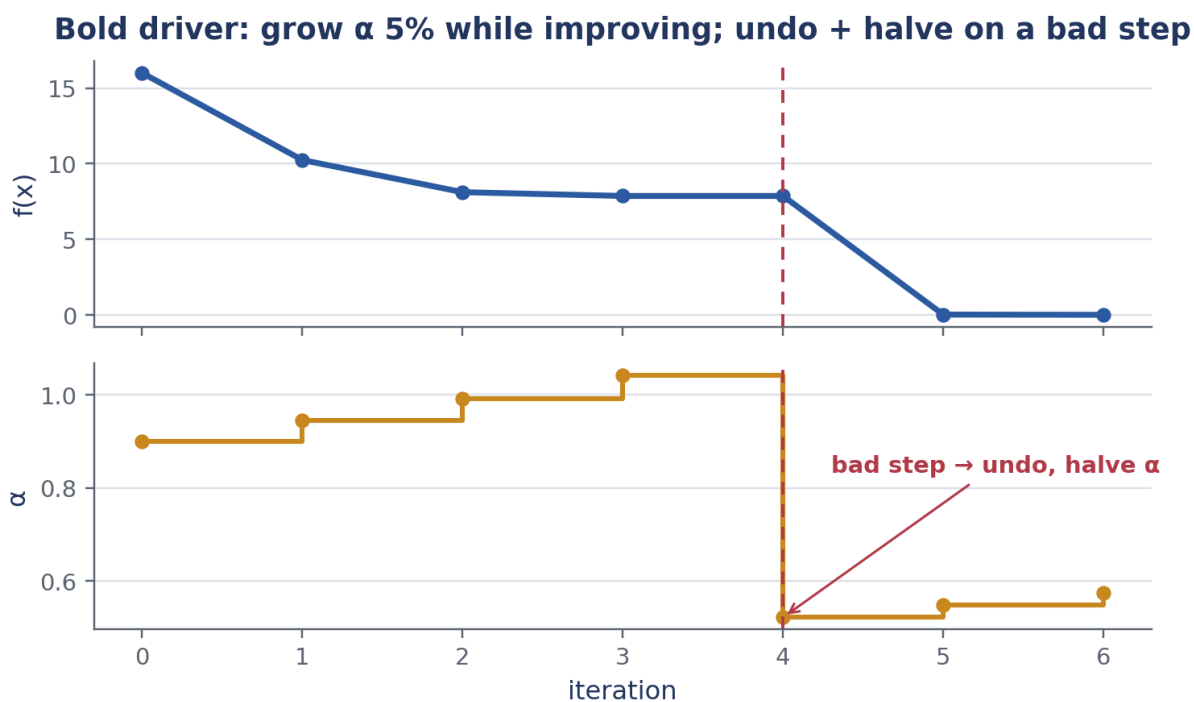


Figure 10: The run above. How to read it: the top panel is the kept loss per iteration; the bottom panel is the learning rate creeping up 5% per win, then halving at the dashed red crash line.

#### ⚠ Watch out

On noisy mini-batch losses, raw bold driver misfires: it halves the rate on unlucky batches and grows it on lucky ones. Use the slide's  $m$ -step version, and cap how large  $\alpha$  may grow.

#### ✓ Key takeaway

Bold driver: improvement  $\Rightarrow$  keep the step and grow  $\alpha$  by  $\sim 5\%$ ; worsening  $\Rightarrow$  undo the step and halve  $\alpha$ . Cheap, self-tuning, and safe — as long as the loss readings are trustworthy.

## 10 Line search: steepest descent and the 90-degree rule

Section 6 found the perfect step by a formula, which only quadratics allow. **Line search** is the general version: pick a direction, then *search* along that line for the step that minimises the objective. The slides note it is rarely used in vanilla gradient descent, because it costs too much per step. But some specialised methods depend on it.

First, names. Let  $J(\mathbf{w})$  be the function being optimised, with parameter vector  $\mathbf{w}_t$  at the start of step  $t$ . The chosen **descent direction** is  $\mathbf{g}_t$ . In the **steepest-descent** method it is the straight-downhill choice,  $\mathbf{g}_t = -\nabla J(\mathbf{w}_t)$ . The update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \mathbf{g}_t, \quad \alpha_t = \arg \min_{\alpha} J(\mathbf{w}_t + \alpha \mathbf{g}_t), \quad (15)$$

where  $\arg \min_{\alpha}$  means “the  $\alpha$  value that makes the expression smallest”. Note the plus sign: the downhill direction is already baked into  $\mathbf{g}_t$ .

### The intuition

Slice the landscape along your walking direction and you get a one-variable curve in  $\alpha$ . Line search finds that curve’s lowest point. And at that lowest point, something geometric happens: the new ground tilts exactly sideways to the direction you just walked.

That geometric fact is the slide’s key claim. After a perfect line-search step, the gradient at  $\mathbf{w}_{t+1}$  is **perpendicular** to the search direction  $\mathbf{g}_t$ . Two vectors are perpendicular when their dot product is zero. Here is the slide’s argument, expanded. Suppose the dot product  $\mathbf{g}_t^{\top} \nabla J(\mathbf{w}_{t+1})$  were *not* zero. Consider a small extra move of  $\pm \delta \mathbf{g}_t$ : a tiny amount  $\delta$  further along the same line, or back along it. Taylor’s first-order rule prices that move. The change in  $J$  is about the move dotted with the gradient:

$$J(\mathbf{w}_t + \alpha_t \mathbf{g}_t \pm \delta \mathbf{g}_t) \approx J(\mathbf{w}_t + \alpha_t \mathbf{g}_t) \pm \delta \mathbf{g}_t^{\top} \nabla J(\mathbf{w}_t + \alpha_t \mathbf{g}_t). \quad (16)$$

If that dot product is positive, choose the minus sign. If it is negative, choose the plus sign. Either way we find a nearby point on the line with a *smaller*  $J$  — contradicting the claim that  $\alpha_t$  was the line’s minimum. So at the true minimum, the dot product must be zero: perpendicular. The same fact drops out of the chain rule in one line. The slope of  $h(\alpha) = J(\mathbf{w}_t + \alpha \mathbf{g}_t)$  is  $h'(\alpha) = \mathbf{g}_t^{\top} \nabla J(\mathbf{w}_t + \alpha \mathbf{g}_t)$ . A minimum of  $h$  needs  $h'(\alpha_t) = 0$ .

### Worked example: verifying the 90-degree rule with Section 6’s numbers

On  $J(x, y) = x^2 + 3y^2$  we took a perfect step from  $(2, 2)$ .

**Step 1. Search direction.**  $\mathbf{g}_0 = -\nabla J(2, 2) = -(4, 12) = (-4, -12)$ .

**Step 2. Landing point.** Section 6 computed  $\mathbf{w}_1 = (1.2857, -0.1429)$ .

**Step 3. Gradient at the landing point.**  $\nabla J(\mathbf{w}_1) = (2x, 6y) = (2.5714, -0.8571)$ .

**Step 4. Dot product.**  $(-4)(2.5714) + (-12)(-0.8571) = -10.2857 + 10.2857 = 0$ .

**Step 5. Sense-check.** Exactly zero, not just small: the perfect step really does land where the new gradient is perpendicular to the old direction. That is why the path in the figure turns square corners.

Successive directions meeting at right angles is precisely the zig-zag we saw in the quadratic’s narrow valley.

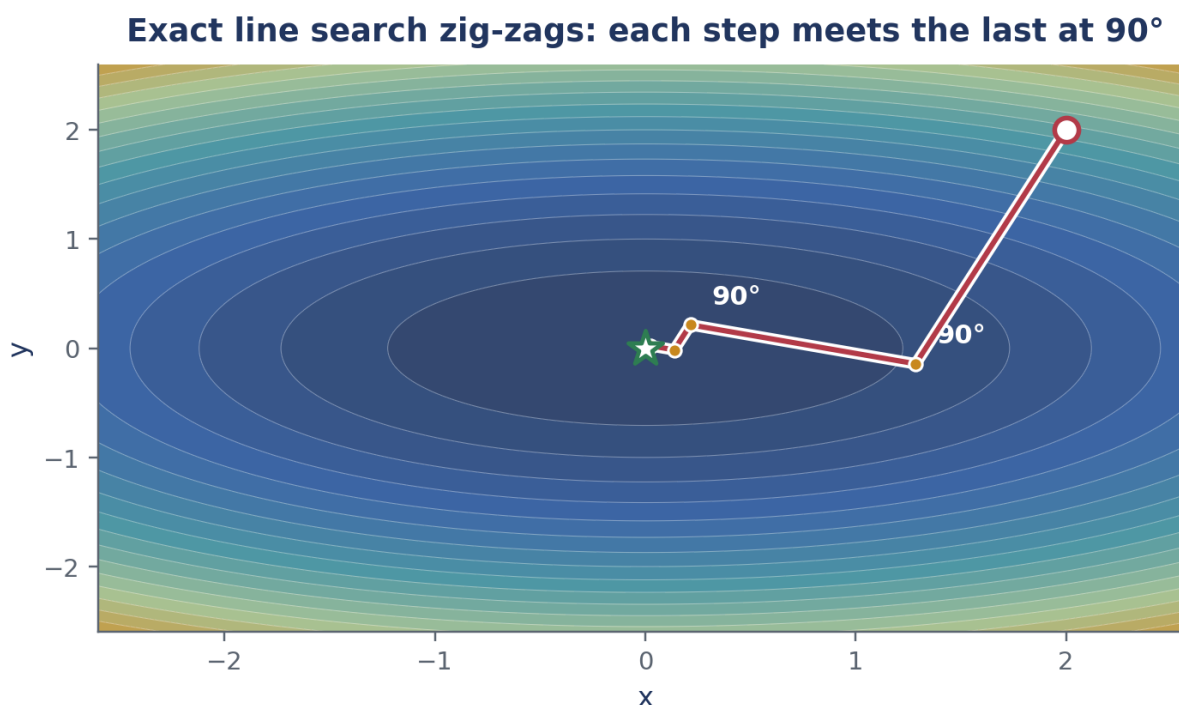


Figure 11: Exact line search on  $x^2 + 3y^2$  from  $(2, 2)$ . How to read it: every segment is one perfect step; each new segment leaves at 90 degrees from the last, so the path staircases into the star at the origin.

One question remains: how do we actually minimise over  $\alpha$ ? The slides lean on a helpful property. In typical line-search settings, the objective is a **unimodal** function of  $\alpha$ . Along the line it falls, bottoms out once, and rises — no second dip. This holds especially when the search uses a quadratic or convex approximation of the objective. The plan is then: first find a range  $[0, \alpha_{\max}]$  that surely contains the dip, then narrow it. To find the range, evaluate the objective at geometrically growing values of  $\alpha$ . Each probe is a fixed multiple of the last. Stop when the value rises.

**Worked example: bracketing the dip of  $h(\gamma)$  by a geometric sweep**

Sweep Section 6's first-step curve  $h(\gamma) = 4(1 - 2\gamma)^2 + 12(1 - 6\gamma)^2$  at  $\gamma = 0.001, 0.01, 0.1, 0.5$ .

**Step 1. Evaluate the sweep.**  $h(0.001) = 15.84$ ;  $h(0.01) = 14.44$ ;  $h(0.1) = 4.48$ ;  $h(0.5) = 48$ .

**Step 2. Spot the turn.** The values fall, fall, fall — then jump up at 0.5. A unimodal curve that has risen cannot dip again.

**Step 3. Declare the bracket.** The minimum lies before the rise:  $[0, \alpha_{\max}] = [0, 0.5]$ .

**Step 4. Sense-check.** Section 6's exact answer  $\gamma^* = 0.1786$  indeed sits inside  $[0, 0.5]$ .

Four cheap evaluations bought a guaranteed playground for the next two sections.

Within the bracket, the slides name three narrowing tools. **Binary search** and **golden-section search** are exact methods, and both need unimodality (Sections 11 and 12). The **Armijo rule** is an inexact method. It does not rely on unimodality, so it applies more broadly.

**Where this shows up in ML.** Steepest descent plus exact line search is the textbook base case of classical optimisation. Its right-angle zig-zag motivates conjugate-gradient methods. They deliberately bend the next direction to stop the staircase.



**✗ Watch out**

The 90-degree rule holds only at the *exact* line minimum. Stop the search early and the property fails. And without unimodality, interval methods can confidently zoom in on the wrong dip.

**✓ Key takeaway**

Line search: move along  $\mathbf{g}_t$  to the  $\alpha$  that minimises  $J$ . At that point the new gradient is perpendicular to  $\mathbf{g}_t$ . Finding  $\alpha_t$  then reduces to trapping the dip of a one-variable unimodal curve in a bracket  $[0, \alpha_{\max}]$ .

## 11 Binary search: halve the bracket every round

We hold a bracket  $[a, b] = [0, \alpha_{\max}]$  that contains the single dip. Binary search shrinks it by asking one question at the midpoint: *is the curve rising or falling here?*

### The intuition

On a one-dip curve, the slope at any point is a signpost. Rising means the dip is behind you (to the left); falling means it is ahead (to the right). Read the signpost at the middle and you may throw away half the interval, every single round.

### ★ Everyday picture

To find a word in a paper dictionary, you open to the middle. You check one thing: am I past it or before it? Then you ignore half the book. Binary search on the step size is the same move, with “alphabetical order” replaced by “which way the curve tilts”.

How do we read the tilt without a derivative formula? The slide’s trick: evaluate the objective at two points squeezed close together. Use the midpoint  $\frac{a+b}{2}$  and its neighbour  $\frac{a+b+\epsilon}{2}$ . Here  $\epsilon$  (epsilon) is tiny, around  $10^{-6}$ . If the second value is larger, the curve is increasing at the midpoint, so the dip lies left: shrink to  $[a, \frac{a+b+\epsilon}{2}]$ . Otherwise it is decreasing, the dip lies right: shrink to  $[\frac{a+b}{2}, b]$ . Repeat.

### Worked example: four rounds on $h(\gamma)$ , bracket $[0, 0.5]$

Recall  $h(\gamma) = 4(1 - 2\gamma)^2 + 12(1 - 6\gamma)^2$ , with its true dip at  $\gamma^* = 5/28 \approx 0.1786$ .

**Step 1. Round 1.** Midpoint 0.25:  $h(0.25) = 4.0000$ , and the probe  $h(0.250001)$  is a shade *larger* — increasing. Dip is left: new bracket  $[0, \mathbf{0.25}]$ .

**Step 2. Round 2.** Midpoint 0.125:  $h(0.125) = 3.0000$ , probe smaller — decreasing. Dip is right:  $[\mathbf{0.125}, 0.25]$ .

**Step 3. Round 3.** Midpoint 0.1875:  $h = 1.7500$ , increasing.  $[0.125, \mathbf{0.1875}]$ .

**Step 4. Round 4.** Midpoint 0.15625:  $h = 1.9375$ , decreasing.  $[\mathbf{0.15625}, 0.1875]$ .

**Step 5. Tally.** Four rounds cut the width from 0.5 to 0.03125 — a factor of  $2^4 = 16$  — and  $\gamma^* = 0.1786$  still sits inside.

**Step 6. Sense-check.** Every kept interval contains the true dip, and each round used just two function evaluations.

Ten more rounds would pin  $\gamma^*$  to about five decimal places; the cost grows only linearly while the precision doubles per round.

**Where this shows up in ML.** Interval halving on a one-dimensional response is everywhere in practice. Learning-rate range probes reuse this loop. So do threshold tuning and bisection on regularisation strength.

### ✗ Watch out

The probe gap  $\epsilon$  must beat floating-point noise, or the “rising or falling” verdict becomes a coin flip. And the whole method leans on unimodality: with two dips, discarding a half can discard the better dip.

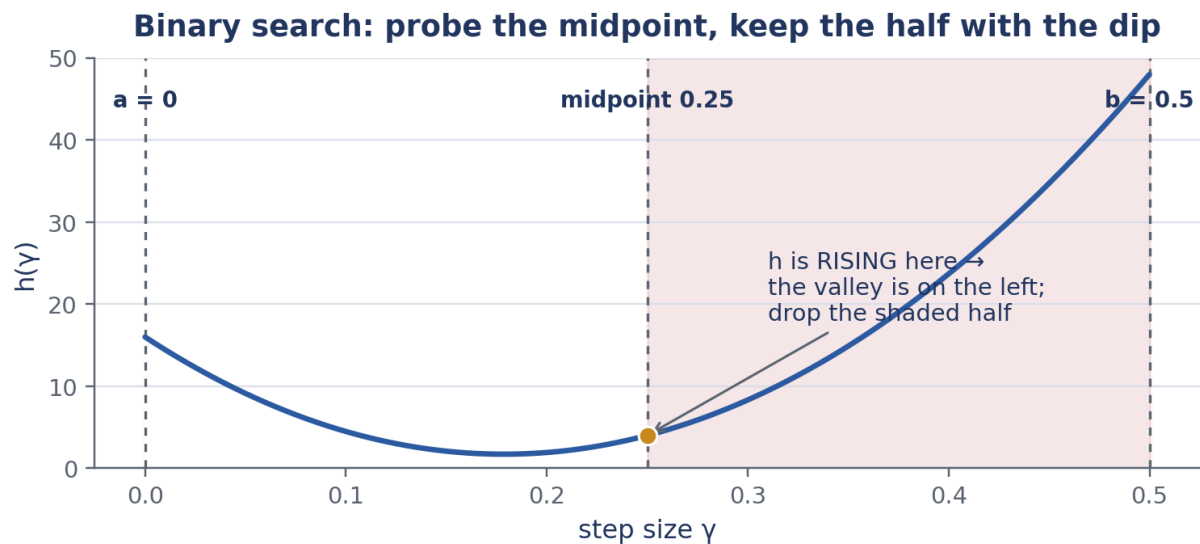


Figure 12: Round 1 of the table. How to read it: at the midpoint the curve tilts upward, so the dip cannot hide in the shaded right half — discard it and repeat on the left.

✓ Key takeaway

Probe the midpoint's tilt with two close evaluations; keep the half that holds the dip. Each round costs two evaluations and halves the bracket.

## 12 Golden-section search: drop the doomed piece

Binary search reads a *slope*. Golden-section search compares *values*. It keeps two interior sample points and discards the stretch of interval that provably cannot contain the minimum.

### The intuition

With the ends and two interior points evaluated, look at where the smallest value sits. On a one-dip curve, the dip must live next to the smallest sample. Whatever lies beyond a larger neighbour is doomed territory — cut it off.

The setup, as on the slide: initialise  $[a, b] = [0, \alpha_{\max}]$  and pick two mid-samples  $m_1, m_2$  with  $a < m_1 < m_2 < b$ . Evaluate the objective  $H$  at all four points. The slide's dropping rules, case by case:

- smallest value at  $\alpha = a$ : drop  $(m_1, b]$ .
- smallest value at  $\alpha = b$ : drop  $[a, m_2)$ .
- smallest value at  $\alpha = m_1$ : drop  $(m_2, b]$ .
- smallest value at  $\alpha = m_2$ : drop  $[a, m_1)$ .

Sometimes both end-pieces go at once. Reset the interval to what survives, and note which evaluated points fall inside it.

The follow-up slide handles the bookkeeping. Suppose the surviving interval contains *no* evaluated point. Then pick a fresh point  $p$  at random inside it, and add a second point  $q$  in the larger of the two pieces  $p$  creates. If instead it contains exactly *one* evaluated point  $p$ , add  $q$  in the larger of  $[a, p]$  and  $[p, b]$ . Either way you again hold four points: two ends, two interior. The game repeats until the interval is as tight as you need.

### Worked example: two rounds on $h(\gamma)$ , bracket $[0, 0.5]$

Choose the thirds  $m_1 = 1/6$  and  $m_2 = 1/3$  as the first mid-samples.

**Step 1. Evaluate all four.**  $h(0) = 16$ ;  $h(1/6) = 4(2/3)^2 + 12(0)^2 = 16/9 = \mathbf{1.778}$  (the second term dies because  $1 - 6 \cdot \frac{1}{6} = 0$ );  $h(1/3) = 4(1/3)^2 + 12(-1)^2 = \mathbf{12.444}$ ;  $h(0.5) = 48$ .

**Step 2. Apply the rule.** The smallest value sits at  $m_1$ . Rule: drop  $(m_2, b]$ . Survivor:  $[0, 1/3]$ , width 0.333, holding one evaluated point,  $p = 1/6$ .

**Step 3. Re-seed.** The pieces  $[0, 1/6]$  and  $[1/6, 1/3]$  are equal, so take  $q$  in the right piece:  $q = 0.25$ ,  $h(0.25) = 4$ .

**Step 4. Round 2.** Four points again: 0 (16),  $1/6$  (1.778), 0.25 (4),  $1/3$  (12.444). Smallest at the first interior point  $1/6$ : drop  $(0.25, 1/3]$ . Survivor:  $[0, \mathbf{0.25}]$ .

**Step 5. Sense-check.** Widths ran  $0.5 \rightarrow 0.333 \rightarrow 0.25$ , and the true dip  $\gamma^* = 0.1786$  survived both cuts, as the rules guarantee for a unimodal curve.

Value comparisons alone — no slopes, no probes — steadily corner the minimum.

Why “golden”? Place the interior points a fraction  $(\sqrt{5} - 1)/2 \approx 0.618$  of the width from each end — the golden ratio. Then after every cut, one old interior point already sits where the next round needs it. Each round then costs only *one* new evaluation, against binary search's two

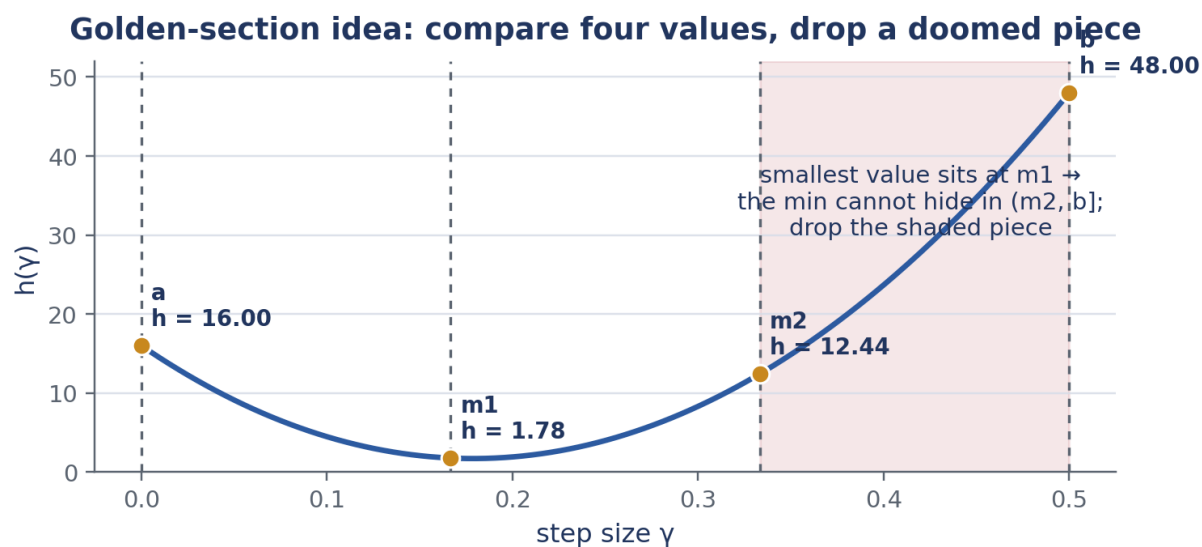


Figure 13: Round 1 of the table. How to read it: four evaluated points; the smallest sits at  $m_1$ , so on a one-dip curve nothing beyond  $m_2$  can win — the shaded piece is discarded whole.

probes.

**Where this shows up in ML.** Golden-section search and its parabolic cousin, Brent's method, are value-only minimisers. They power the `minimize_scalar` routine inside scientific Python. They turn up wherever one hyperparameter must be tuned by evaluations alone.

#### ⚠ Watch out

The dropping rules are proofs, not hunches — but only under unimodality. Feed the method a two-dip curve and it will happily discard the better dip and polish the worse one.

#### ✓ Key takeaway

Keep four evaluated points; the smallest value tells you which stretch cannot contain the minimum; drop it and refill. Golden-ratio spacing recycles a point so each round costs one evaluation.

### 13 When is line search worth the cost?

We now own an expensive precision tool. The last line-search slide asks the practical question: when should we pay for it? Its verdict, unpacked: line search provably converges to a local optimum. But each of its steps costs many function evaluations. That is why vanilla gradient descent rarely uses it. Some methods, though, have no choice — Newton’s method needs an exact line search to behave. For everyday descents, fast *inexact* rules like Armijo’s are the compromise. Try a step. If the loss did not fall by at least a set fraction of what the slope promised, shrink and retry. A couple of tries usually suffice.

#### The intuition

Count the true bill: total work equals work per step times number of steps. A perfect step is expensive but you need few of them. A cheap step is nearly free but you need many. Neither side wins by default; the product decides.

#### ★ Everyday picture

Crossing a city, you can stop and study the map at every corner: slow corners, few wrong turns. Or you can stride on instinct: instant corners, several wrong turns. The fastest trip is usually a glance at the map only when the road looks confusing — the inexact compromise.

#### Worked example: a stylised bill for three step-size policies

Suppose one descent must cross the same valley three ways. The numbers below are an illustration of the trade, not a benchmark.

**Step 1. Fixed step.** Cheap steps: 1 evaluation each, but cautious, so say 400 steps. Bill:  $400 \times 1 = \mathbf{400}$  evaluations.

**Step 2. Armijo (inexact search).** About 2 evaluations per step (one try, sometimes a retry), and better steps mean fewer of them, say 150. Bill:  $150 \times 2 = \mathbf{300}$ .

**Step 3. Exact line search.** Each step runs a Section 11/12 search, say 12 evaluations, but the steps are excellent, say 40 of them. Bill:  $40 \times 12 = \mathbf{480}$ .

**Step 4. Read the moral.** The middle policy wins here, and that matches practice. Exact search pays only when steps themselves are precious. That is the slide’s point: fewer steps “might more than compensate” the cost of each.

Change the ratios and a different column wins; the product, not either factor, is the decision.

**Where this shows up in ML.** Deep-learning training loops choose fixed schedules — per-step line search across billions of weights and noisy mini-batches is unaffordable. Classical solvers keep a line search, usually an inexact one (the BFGS family, conjugate gradient). Newton-style methods still need the exact kind.

#### ✗ Watch out

Comparing optimisers by iteration counts alone is a classic trap. A method with half the steps and ten times the per-step work is five times slower. Always price the whole product.

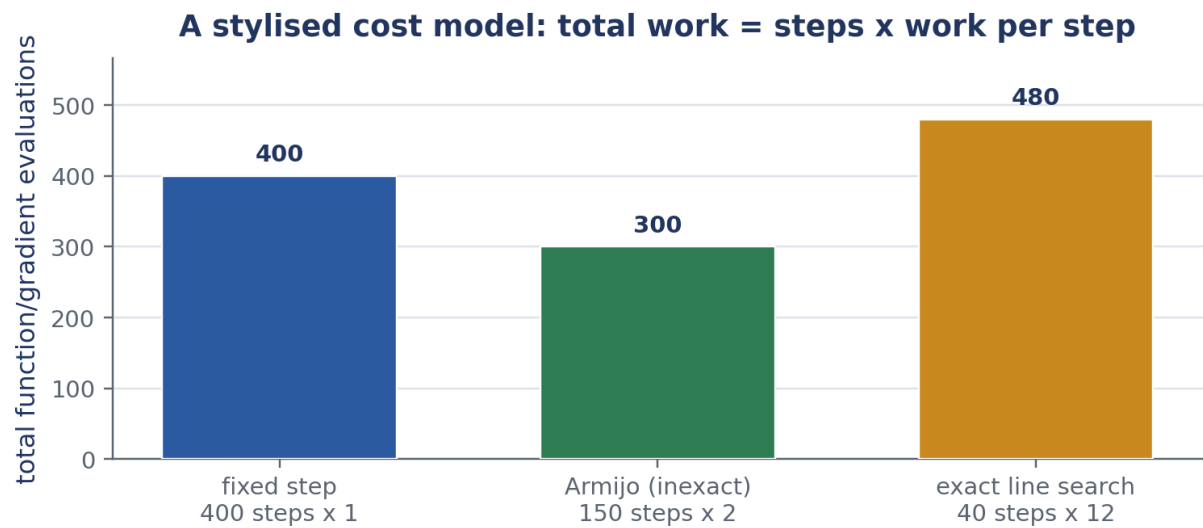


Figure 14: The stylised bill as a picture. How to read it: each bar is total evaluations, the product of steps taken and evaluations per step; the cheapest bar belongs to the inexact middle ground.

✓ **Key takeaway**

Exact line search: reliable but expensive per step — reserve it for methods that demand it (Newton) or steps that are precious. Otherwise, an inexact Armijo-style rule or a plain schedule gives more descent per unit of compute.

## 14 Stochastic gradient descent: learning from a spoonful

Now the machine-learning finale. A training loss is a sum over data points. Each point  $n$  contributes its own loss  $L_n(\theta)$ , and the total is

$$L(\theta) = \sum_{n=1}^N L_n(\theta). \quad (17)$$

Here  $\theta$  (theta) is the parameter vector we are tuning.  $N$  is the number of data points. The goal of learning is the  $\theta$  that minimises  $L$ . Standard gradient descent is a **batch** method. Its update uses the gradient of the *entire* sum:

$$\theta_{i+1} = \theta_i - \gamma_i (\nabla L(\theta_i))^\top = \theta_i - \gamma_i \sum_{n=1}^N (\nabla L_n(\theta_i))^\top. \quad (18)$$

One step reads all  $N$  points. With  $N$  in the millions, one step is a full pass over the data — far too expensive.

### The intuition

You do not need every data point to know which way is downhill. A random handful of points gives a gradient that is roughly right. Roughly right is enough when you take many cheap steps. Trade perfection per step for a mountain of steps.

### ★ Everyday picture

To season a huge pot of soup you do not drink the pot; you stir and taste a spoonful. The spoon misjudges a little every time, but repeated taste-and-adjust rounds land the seasoning. Stir first — the spoon must be a random sample. The analogy's break point: a spoonful is almost free, but noisy. So you must keep your adjustments small near the end.

Formally, let the data be  $\mathbf{X}_1, \dots, \mathbf{X}_n$  with targets  $y_i$ , and let  $S$  be a subset of the indices  $\{1, 2, \dots, n\}$ . Treat  $S$  as a sample. For a linear model with weights  $\mathbf{w}$ , the slide builds the sample-only objective

$$J(S) = \sum_{i \in S} (\mathbf{w}^\top \mathbf{X}_i - y_i)^2. \quad (19)$$

Now the key idea of **stochastic gradient descent**. The gradient of  $J(S)$  with respect to  $\mathbf{w}$  is an excellent stand-in for the true gradient. The update swaps it in, component by component:

$$\begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} \leftarrow \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} - \alpha \begin{bmatrix} \partial J(S)/\partial w_1 \\ \vdots \\ \partial J(S)/\partial w_d \end{bmatrix}. \quad (20)$$

Vocabulary time. A moderately sized  $S$  gives **mini-batch** gradient descent. The extreme  $S$  of one random index is stochastic gradient descent proper. Why does a subset work at all? Because the sampled gradient is an **unbiased** estimate of the true one. On average over the random draw, it equals exactly the full-data gradient. Errors on single draws cancel out over many steps. The slides add the theory headline. Let the learning rate decrease at a suitable rate, and let



mild assumptions hold. Then SGD converges **almost surely** — with probability one — to a local minimum. Batch size sets the noise level. Gradients from large mini-batches have lower **variance** — less step-to-step scatter — than gradients from small ones.

 **Worked example: every per-point gradient of our line-fit problem, at  $w = (0, 0)$**

Use the Section-1 data with the intercept trick  $\mathbf{X}_i = (x_i, 1)$  and  $\mathbf{w} = (a, b)$ , so  $\mathbf{w}^\top \mathbf{X}_i = ax_i + b$ . One point's gradient is  $\nabla J(\{i\}) = 2(\mathbf{w}^\top \mathbf{X}_i - y_i) \mathbf{X}_i$ .

**Step 1. Point 1** ( $x=1, y=3.1$ ): miss =  $0 - 3.1 = -3.1$ ; gradient =  $2(-3.1)(1, 1) = (-6.2, -6.2)$ .

**Step 2. Point 2** ( $2, 4.9$ ):  $2(-4.9)(2, 1) = (-19.6, -9.8)$ .

**Step 3. Point 3** ( $3, 7.3$ ):  $2(-7.3)(3, 1) = (-43.8, -14.6)$ .

**Step 4. Point 4** ( $4, 9.1$ ):  $2(-9.1)(4, 1) = (-72.8, -18.2)$ .

**Step 5. Full-batch gradient.** The sum:  $(-6.2 - 19.6 - 43.8 - 72.8, -6.2 - 9.8 - 14.6 - 18.2) = (-142.4, -48.8)$ .

**Step 6. See the unbiasedness.** Draw one index uniformly and scale its gradient by  $N=4$ . The four possible estimates average to  $\frac{4}{4}[(-6.2) + (-19.6) + (-43.8) + (-72.8)] = -142.4$  in the first slot. The second slot likewise averages to  $-48.8$ : exactly the full gradient. Right *on average* — that is the definition of unbiased.

**Step 7. Sense-check.** Every per-point gradient has both entries negative. So each sample, alone, tells  $a$  and  $b$  to grow — the correct downhill call for this loss.

No single spoonful equals the pot, but their average is the pot, exactly.

How wrong is one spoonful? Measure the angle between each sampled gradient and the true one. Single points miss by  $26.1^\circ$ ,  $7.7^\circ$ ,  $0.5^\circ$  and  $4.9^\circ$  — average  $9.8^\circ$ . Two-point batches average  $4.3^\circ$ . The full batch is exact by definition. Bigger spoonfuls, straighter aim — the slide's variance claim made visible.

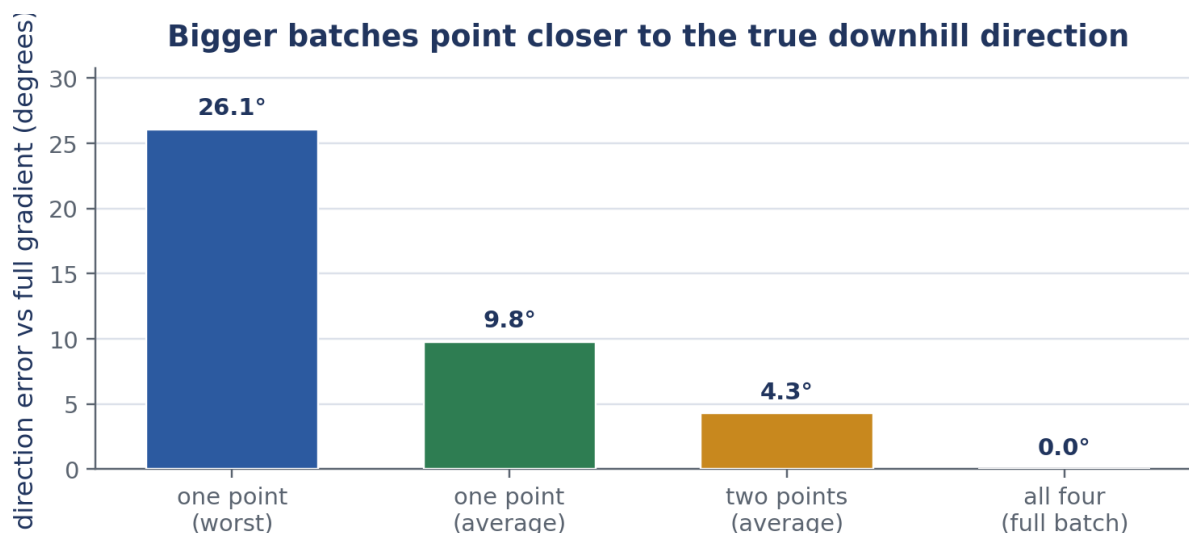


Figure 15: Sampled-gradient direction error on our four-point problem. How to read it: each bar is the angle between a sampled gradient and the true one; the error shrinks as the batch grows, which is exactly “larger mini-batches have lower variance”.

**Where this shows up in ML.** This section is the algorithm that trains most of modern AI:

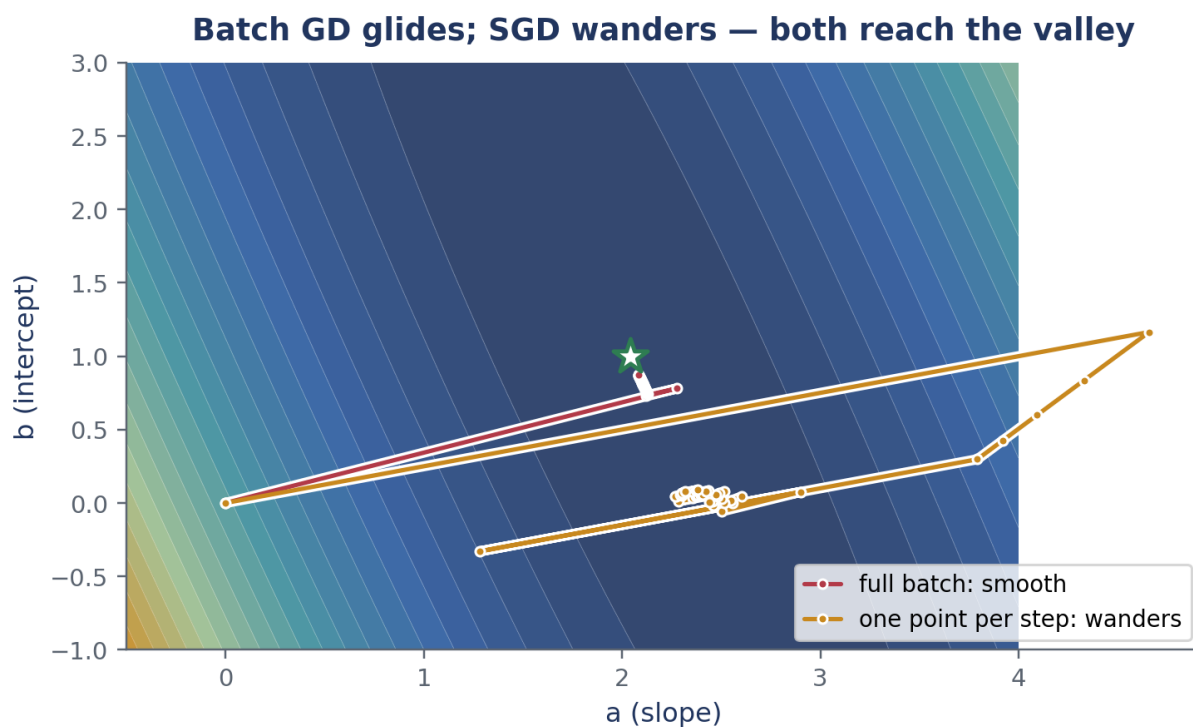


Figure 16: Two descents of the same loss  $L(a, b)$ . How to read it: the smooth trail is full-batch gradient descent; the jagged trail re-samples one random point per step, so it wanders — yet with a decaying rate it still finds the star.

language models, vision models, recommenders. The vocabulary here — epoch, mini-batch, batch size, schedule — is the daily dialect of deep learning. Adam, the default optimiser, is SGD wearing per-weight step sizes.

#### ⚠ Watch out

SGD's noise never dies on its own: with a constant rate the walk jitters around the minimum forever. The convergence promise needs a *decreasing* rate (Section 8's schedules). And judge progress over many steps, not one — a single noisy step says almost nothing (Section 9's  $m$ -step rule).

#### ✓ Key takeaway

Replace the full gradient with the gradient of a random sample: unbiased, vastly cheaper, noisier. Small batches buy speed with variance; a decaying learning rate spends that variance down until SGD settles into a local minimum.

## Glossary & symbols

---

### Key terms.

|                                   |                                                                           |
|-----------------------------------|---------------------------------------------------------------------------|
| <b>loss function</b>              | the badness score we minimise; here, the sum of squared gaps.             |
| <b>unconstrained optimization</b> | minimising with no side rules on the variables.                           |
| <b>gradient</b>                   | the vector of all partial slopes; points steepest uphill.                 |
| <b>stationary point</b>           | a place where the gradient is zero: valley, hilltop, or saddle.           |
| <b>second-derivative test</b>     | positive curvature means minimum; negative means maximum.                 |
| <b>local / global minimum</b>     | lowest nearby versus lowest anywhere.                                     |
| <b>gradient descent</b>           | repeat: step against the gradient, scaled by the step size.               |
| <b>step size / learning rate</b>  | how far one step travels along the downhill direction.                    |
| <b>first-order method</b>         | uses slopes only, never curvatures; that is why it scales.                |
| <b>monotonic convergence</b>      | the recorded loss never rises on the way down.                            |
| <b>finite difference</b>          | slope estimated as (change in output)/(small nudge in input).             |
| <b>learning-rate decay</b>        | shrinking the rate over time: exponential, inverse, or step.              |
| <b>epoch</b>                      | one full cycle through the training data.                                 |
| <b>bold driver</b>                | grow the rate 5% on improvement; undo the step and halve it on worsening. |
| <b>line search</b>                | minimise the objective along a fixed direction to pick the step.          |
| <b>steepest descent</b>           | the line-search method whose direction is the negative gradient.          |
| <b>unimodal</b>                   | falls to one dip and rises; no second valley on the interval.             |
| <b>bracket</b>                    | an interval $[0, \alpha_{\max}]$ guaranteed to contain the dip.           |
| <b>binary search</b>              | halve the bracket by probing the midpoint's tilt.                         |
| <b>golden-section search</b>      | drop the piece of the bracket that provably lacks the dip.                |
| <b>Armijo rule</b>                | inexact line search: accept a step once the drop is good enough.          |
| <b>batch gradient descent</b>     | each update uses the gradient of the whole data sum.                      |
| <b>mini-batch / SGD</b>           | each update uses a random subset — or a single random point.              |

**unbiased estimate**      right on average over the random draw.

**variance (of a gradient estimate)**  
                                  step-to-step scatter; shrinks as batches grow.

**Symbols at a glance.**

| Symbol                                 | Meaning                                                  |
|----------------------------------------|----------------------------------------------------------|
| $a, b$                                 | slope and intercept of the fitted line                   |
| $\hat{y}$                              | the model's predicted value                              |
| $L(a, b), L(\theta)$                   | the loss: total squared gap over the data                |
| $l(x)$                                 | the slide's quartic example function                     |
| $\nabla f$                             | the gradient of $f$ (row vector in this lecture)         |
| $\top$                                 | transpose: stands a row up into a column                 |
| $\mathbf{x}_i$                         | the current point after $i$ descent steps                |
| $\gamma, \gamma_i$                     | the step size (learning rate), possibly per step         |
| $A, \mathbf{b}$                        | the matrix and vector of the quadratic bowl example      |
| $\mathbf{x}_*$                         | the true minimiser, where $A\mathbf{x}_* = -\mathbf{b}$  |
| $h(\gamma)$                            | loss along one step direction, as a function of $\gamma$ |
| $J(\mathbf{w})$                        | objective in the line-search and SGD sections            |
| $\mathbf{w}_t, \mathbf{g}_t, \alpha_t$ | parameters, search direction, step at step $t$           |
| $\alpha_0, k, t$                       | starting rate, decay speed, epoch counter                |
| $\Delta$                               | the small nudge in the finite-difference check           |
| $\epsilon$                             | the tiny probe gap in binary search ( $\sim 10^{-6}$ )   |
| $\theta$                               | the parameter vector being learned                       |
| $L_n, N$                               | one data point's loss; the number of data points         |
| $S$                                    | the sampled subset of data indices (the mini-batch)      |
| $\mathbf{X}_i, y_i$                    | one data point's features and target                     |